Federated Object Detection Under Extreme Non-IID Labels in Biomedical Imaging: System Proposal and Analysis of Reliability and Robustness

Salvatore Capuozzo [1*], Stefano Marrone [2] and Carlo Sansone [2]

[1*]Department of Industrial Engineering, University of Naples Federico II, Piazzale Tecchio 80, Naples, 80125, Italy.
[2]Department of Electrical Engineering and Information Technologies, University of Naples Federico II, Via Claudio 21, Naples, 80125, Italy.

*Corresponding author(s). E-mail(s): salvatore.capuozzo@unina.it;
Contributing authors: stefano.marrone@unina.it;
carlo.sansone@unina.it;

Abstract

Medical object-detection data are siloed by site, and sites often annotate different findings. Federated learning can train one detector across such sites without moving images, but trusting the result requires knowing, with stated confidence, how much accuracy federation costs, who bears that cost, and how the system behaves when a participant misbehaves. We present COFY (Class-Owned Federated YOLO), a YOLO detector for nodes that own disjoint sets of classes, in which each class's output parameters come only from their owner and the server holds no data. It supports parallel averaging (FedAvg), sequential turns (cyclic weight transfer, CWT) and concurrent asynchronous updates (FedAsync), the latter with barrier-free calibration. Accuracy relative to a centralized model trained on the same partially labelled data is framed as a non-inferiority question: across ten fresh training seeds, with code, configurations and analysis frozen and hashed before the first run, we report the lower bound of the 95% confidence interval of the federated-to-centralized mAP50 ratio on every dataset. On parasite eggs, white blood cells and dental radiographs, CWT retained at least 95% of centralized mAP50 (lower bounds 96.2–98.7%) and FedAsync at least 90%. Per-class analysis shows that averaging-based schemes concentrate losses on rare classes, a disparity hidden by mean metrics. In a smaller robustness study, ownership-aware robust aggregation neutralized update scaling, weight inflation

and class-row hijacking, but neither a backdoor in the attacker's own class nor a node reversing its updates, which collapsed every scheme. Follow-up tests point to remedies that remain to be confirmed.

Keywords: Federated learning, Object detection, Trustworthy AI, Non-inferiority, Asynchronous federated learning, Byzantine robustness, Medical imaging, Reproducibility

1 Introduction

Federated learning (FL) trains a shared model on data that stay where they were collected [1, 2]. In medicine this matters because images cannot leave hospitals and laboratories easily, and models trained at a single site generalize poorly [3–6]. Most medical FL work addresses classification or segmentation with the same label set at every site. Object detection in real networks is different. Sites annotate what they see and what they are expert in, so the label sets themselves differ. A veterinary parasitology laboratory may label the helminth eggs of the species it diagnoses. A haematology service may label some cell types and not others. A radiology department may annotate a subset of findings. The same image can contain objects that one site would label and another would ignore. Training a single detector for all classes then means learning from nodes that each own a different subset of classes, and that treat the other classes as unlabelled background. This is label-distribution skew in its most extreme form: no class is labelled at more than one node, and in detection the same images can also contain the other nodes' objects, unlabelled.

In our preliminary experiments, an asynchronous, class-aligned federation of YOLO detectors covered all classes in such a network of microscopy laboratories, but it still lagged behind a centralized model. This raises the question of trust that motivates the present work. Before deploying a federated detector instead of pooling data, practitioners and regulators need four answers. How much accuracy does federation cost, and how sure are we? Is the cost shared fairly across classes and sites? What does each operating mode trade off: waiting time, privacy guarantees, robustness? What happens if a participant is faulty or malicious? A single comparison of mean mAP on one seed answers none of these. Federated detection results vary with the seed, the arrival order of updates and the training endpoint as much as with the method [7]. Methods are also usually selected on the same runs that are then reported.

This paper answers these questions for federated detection with disjoint class ownership. Its contributions are:

1. **A system for disjoint class ownership.** COFY (Class-Owned Federated YOLO) is a federated detector for nodes that own disjoint class sets, built on a decoupled YOLO head. What is new is the ownership rule and what it requires. The head is split into shared parameters and per-class rows; every aggregation scheme takes each class's rows only from the class's owner; and a barrier-free *progressive calibration* re-fits BatchNorm statistics and class rows without breaking asynchrony. The other components are adapted from prior work and combined

under this rule. The loss masks unowned classes, as partial-label methods do for classification and segmentation [8, 9]. The merge rules are FedAvg, cyclic weight transfer (CWT) and FedAsync [1, 10, 11]. The calibration builds on BatchNorm re-estimation and classifier re-fitting [12, 13]. These schemes have been used for detection only when every site labels the same classes; to our knowledge, this is their first use, and the first federated detector, under disjoint class ownership. A secured client–server implementation runs both schemes we evaluate; it has been exercised on one machine, not across sites.

2. **A trustworthy evaluation protocol**, which is, with the ownership rule, the main methodological contribution. The cost of federation is a non-inferiority question, as for a cheaper treatment in clinical trials [14]. It is answered by the lower confidence bound of the federated-to-centralized ratio across training seeds, on every dataset (an intersection-union test [15]). The two schemes are run once, on ten seeds not used to design them, with code, configurations and the analysis script frozen and hashed before the first run. For each scheme we state the highest share of centralized accuracy, in steps of 5%, that its lower bound exceeds on every dataset, and we report the seed-level results.
3. **Bounds on the cost of federation** for synchronous and asynchronous operation on medical detection use cases: CWT keeps at least 95% of centralized mAP50 on KFM12, WBC and DENTEX, with lower bounds of 96.2–98.7%, and FedAsync at least 90%, a bound that hides its rare-class losses. The bounds describe variation across training seeds on these datasets, not across new sites or populations.
4. **Fairness and cost analyses** showing that schemes which average models trained in parallel concentrate their losses on rare classes. We also report the compute and energy of each operating mode, measured on one GPU, its communication, computed from the model size, and its waiting time, derived from a timing model.
5. **Ownership-aware robust aggregation** against malicious or faulty nodes, evaluated in a smaller study (one use case, three seeds, one non-colluding attacker) with detection-specific attacks (box corruption, phantom-object backdoor, BatchNorm poisoning, class-row hijacking). Standard Byzantine-robust rules do not carry over to this setting, because every class has a single owner. We also report where the defence fails: single ownership is what makes the defence possible, and it is also what a node exploits when it plants a backdoor in its own class or reverses its updates. A per-class detection budget at inference contains the second attack within the attacker's own classes, without repairing the model. Four candidate defences, designed after these results and tested afterwards on the same seeds, point further. In those runs, the calibration step undoes BatchNorm poisoning, pooling its statistics by their median protects it from a node that poisons them, and screening inputs at the nodes catches the backdoor in most runs; bounding consecutive merges does not stop BatchNorm poisoning. Peer validation with per-class rollback finds the class of a node that reverses its updates in every run, without false alarms. Rolling back that class restores FedAvg, and also excluding the node lifts CWT from 31% to 77% of centralized and FedAsync from 17% to 97%.

The rest of the article is organized as follows. Section 2 reviews related work. Section 3 defines the problem and the cost of federation, describes COFY, its deployment and its defence, and presents the use cases. Section 4 describes the experimental design: the training protocol, the non-inferiority study, the secondary analyses and baselines, and the robustness study. Section 5 reports the results. Section 6 discusses the operating points, the alignment with requirements for trustworthy AI, privacy and the limitations of the study, and Section 7 concludes.

2 Related work

This work sits at the intersection of four lines of research: learning from heterogeneous and partially labelled federations, federated object detection, asynchronous federated learning, and the robustness and evaluation practices that make a federated system trustworthy. We review each in turn and point out where the constraints of our setting rule out an existing method.

2.1 Heterogeneous and partially labelled federations

FedAvg [1] averages models trained locally. Its degradation under non-IID data motivated proximal terms (FedProx [16]), control variates (SCAFFOLD [17]), normalized aggregation (FedNova [18]), local BatchNorm (FedBN [19]) and classifier re-calibration (CCVR [13]). Label-distribution skew in its extreme form, where each client holds a single class, was studied for classification by FedAwS [20]. FedRS [8] restricts the logits of the classes a client does not hold, and FedGELA [21] targets partially class-disjoint data with a fixed classifier geometry. In medical imaging, federations whose clients annotate different classes have been trained for classification with label-set mismatch (FedLSM [22]), and for segmentation with partial-label losses (FedMENU [9]), conditional distillation (ConDistFL [23]) or clients with different brain diseases and MRI modalities [24]. All of these address classification or segmentation; in detection, an unlabelled object is a region the loss treats as background unless it is masked. Several of the methods also violate the constraints of our setting: CCVR re-fits the classifier on the server from client feature statistics, FedAwS needs a server-side regularizer, and FedBN keeps BatchNorm local, so no single global detector exists.

2.2 Federated object detection

FedVision [25] and the accompanying real-world datasets [26] brought FedAvg to detection with a shared label set. UltraFlwr [27] integrates Ultralytics YOLO with the Flower platform and aggregates YOLO components selectively, for surgical instrument detection with a shared label set. In medical imaging, FedAvg with weight exchanges between institutions has been used for self-configuring object detection [28], and cyclic weight transfer for lesion detection in head MR images [29]; an evaluation for connected vehicles compares synchronous and asynchronous federated detection [30]. All of them assume that every site labels the same classes. Detectors over label spaces that differ between datasets have been trained centrally, with all the data in one place [31], and federated methods for clients with different class sets target classification (previous

section). Our first use case comes from the Kubic FLOTAC Microscope (KFM) [32], a compact digital microscope for faecal egg counts whose automatic egg detector has been developed through the AI-KFM challenge [33] and a dedicated deep-learning workflow for trematode eggs [34], with part of its data released publicly [35]. Each laboratory that adopts the microscope diagnoses its own parasites, which is the setting this work addresses. To our knowledge, FedAvg, cyclic weight transfer and FedAsync have not been applied to object detection under disjoint class ownership, where every class is labelled at a single site while other sites see its objects unlabelled, and no prior work bounds the accuracy cost of federated detection with confidence guarantees or studies its robustness to malicious participants in this setting.

2.3 Asynchronous federated learning

FedAsync [11] applies each update on arrival with a staleness-dependent weight. FedBuff [36] buffers several updates before a server step. CA2FL [37] calibrates buffered updates with a cache of every client's latest update. Most asynchronous studies simulate arrival times or impose an order. We distinguish *sequential* execution (one node at a time) from *concurrent* execution (nodes overlapping in time), which produce different orders and staleness of updates, and we simulate concurrency with a seeded timing model.

2.4 Robustness and trustworthy evaluation

Byzantine-robust aggregation rules (Krum [38], coordinate-wise median and trimmed mean [39]) assume that honest updates agree, and they are vulnerable to tailored attacks [40]; an experimental comparison found that no rule withstands every attack [41]. Several other defences need a clean dataset on the server (FLTrust [42]). Backdoors can be injected by model replacement [43] and mitigated by norm bounding [44]. Client-side validation detects them through feedback (BaFFLe [45]). Kardam [46] filters asynchronous updates. Surveys cover the field [47]. Detection-specific backdoors that create or remove objects exist for centralized training [48]. Poisoning of federated driving models has been studied on regression tasks [49]. We found no defence designed for federated detection, nor for federations in which each class has a single owner, where no majority exists for a class's parameters.

Trustworthy FL is surveyed in [50], and FederatedTrust [51] proposes a trust score built from pillars that follow the European guidelines [52]. Regulation now requires documented accuracy, robustness, logging and human oversight for high-risk AI [53], and the NIST AI RMF asks for measured, documented risk [54]. In clinical research, a claim that a cheaper treatment is "not much worse" is a non-inferiority claim. It is reported with a pre-specified margin and a confidence bound [14]. Machine learning increasingly recognizes the variance hidden in single-run comparisons [7] and the value of pre-registration [55]. We bring the non-inferiority framing, with confidence bounds on fresh seeds and frozen code, to federated learning.

3 Materials and methods

This section first states the problem: who owns which classes, the rules every scheme must respect, how the cost of federation is measured, and what an attacker can do (Section 3.1). It then describes COFY: the model, local training, the three aggregation schemes and the calibration (Section 3.2). Section 3.3 covers the networked implementation and the defence against malicious nodes, and Section 3.4 the three use cases.

3.1 Problem setting

We define who owns which classes and the rules every scheme must respect, how the cost of federation is measured, and what an attacker can do.

3.1.1 Disjoint class ownership

Let $\mathcal{C} = \{1, \dots, K\}$ be the classes and N the nodes. Node i *owns* $\mathcal{C}_i \subset \mathcal{C}$, with $\bigcup_i \mathcal{C}_i = \mathcal{C}$ and $\mathcal{C}_i \cap \mathcal{C}_j = \emptyset$ for $i \neq j$. It holds images $\mathcal{D}_i$ annotated only for $\mathcal{C}_i$. Objects of other classes can appear in its images unannotated. The goal is one detector f_θ for all K classes. Three rules hold throughout:

R1 nodes learn about other nodes' classes only through the model they download;
R2 the server holds no labelled data and trains nothing: it may only combine weights and statistics;
R3 each class has exactly one owner, and only the owner can change that class's output parameters (a node can own several classes).

3.1.2 The cost of federation

A full-label model is not a fair comparator, because nobody holds full labels. We therefore define the cost of federation against a *centralized* model trained on the union of the nodes' data with exactly the same partial labels. It sees the same examples (each image with one node's partial labels and class mask), as often, with the same budget, batch size and learning-rate schedule, and updates one set of weights directly; Section 4.1 gives the exact correspondence and the differences that remain. For seed s, dataset d and scheme m:

$$r_{m,d,s} = 100 \cdot \frac{\text{mAP50}(\text{federated}_{m,d,s})}{\text{mAP50}(\text{centralized}_{d,s})}. \tag{1}$$

mAP50 is the mean, over the classes, of the average precision at an intersection over union of 0.5, with every class weighted equally. It is computed by the Ultralytics validator (version 8.4.84) on the evaluation set: detections above confidence 0.001, non-maximum suppression at an intersection over union of 0.7, at most 300 detections per image, and precision interpolated at 101 recall points. Every class has at least 16 instances in the evaluation set (Table 2), so every class enters the mean; a class with no correct detection scores 0. The ratio of the two aggregate values is the primary quantity, fixed before the first run, because it is free of scale: it compares use cases

whose centralized mAP50 ranges from 0.50 to 0.96. Per-class retention, the same ratio for a single class, is a secondary quantity (Section 4.3). It is noisier for rare classes, but it shows who bears a cost that the mean can hide.

Both models receive the same post-processing (Section 3.2). Over n seeds, with mean $\bar{r}$ and standard deviation s_r, the scheme is non-inferior at margin δ on dataset d if

$$\mathrm{CI}_{\mathrm{low}} = \bar{r} - t_{0.975,\,n-1}\,\frac{s_r}{\sqrt{n}} \;\geq\; \delta, \tag{2}$$

that is, if the lower end of the two-sided 95% confidence interval exceeds δ. A scheme is non-inferior at δ if (2) holds on *every* dataset. This is an intersection-union test, which needs no multiplicity correction [15].

The interval describes variation across training seeds, with each dataset, its split and its node partition fixed. The seed draws the training subset, the initial class rows, the order and augmentation of the training data, the node order of CWT and the machine speeds of FedAsync. Non-inferiority is therefore a statement about COFY on these use cases, as they were split. It is not a clinical non-inferiority claim, and it does not extend to other sites, populations or acquisition protocols.

3.1.3 Threat model

For robustness we assume one of four nodes is malicious or faulty and does not collude with others. It controls its own data, training and uploads, and knows the protocol and the defence. The server is honest. Transport, authentication and payload validation are already secured (Section 3.3). The attacker may aim to degrade the global model (untargeted), to make the model report its class where a trigger appears (targeted backdoor), or to remain undetected.

3.2 The COFY system

COFY consists of a detector whose parameters are split into shared parameters and per-class rows, a masked local loss, three aggregation schemes and a calibration step. Figure 1 gives an overview.

3.2.1 Model and parameter decomposition

COFY uses an anchor-free YOLO detector [56, 57]. The study uses YOLO26 [58] at scale L, about 25 M parameters at 640 px, with the classic decoupled detection head. In this head, the last 1×1 convolution of the classification branch at each scale maps features to one logit per class. Row c of these convolutions (weights and bias, at every scale) is the only parameter that is specific to class c. We call these the *class rows* $\theta^{(c)}$. Every other parameter is *shared*: backbone, neck, box branch and the rest of the classification branch. Shared parameters are initialized from COCO pre-training [59]; class rows start random.

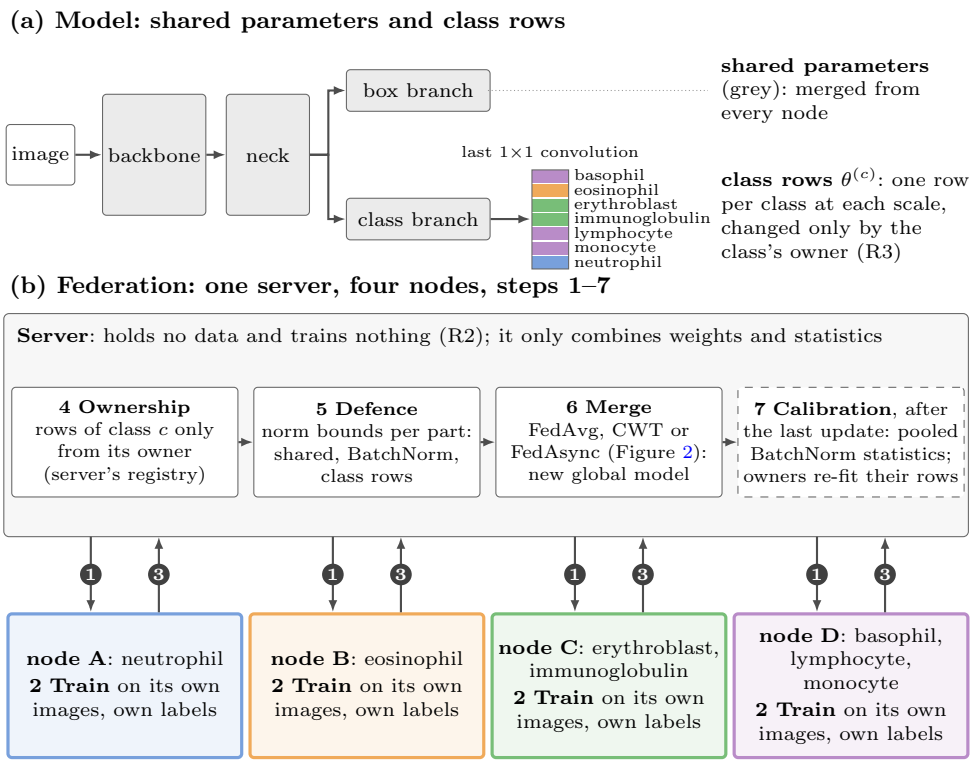


Fig. 1 Overview of COFY. (a) The detector's parameters. All of them are shared and merged from every node, except the class rows: the row of each class in the last 1×1 convolution of the classification branch, at every scale, which only the class's owner changes (the classes of the WBC use case, coloured by their owner). (b) The federation, with its steps numbered in the order they happen. Each node (1) downloads the global model, (2) trains on its own images with the masked loss and (3) uploads its change to the shared parameters and to its own rows. The server, which holds no data, (4) takes each class's rows only from the owner in its registry, (5) bounds each part of the update and (6) merges it into a new global model with FedAvg, CWT or FedAsync (Figure 2); the nodes then start again from (1). After the last update, (7) the calibration pools BatchNorm statistics and lets each owner re-fit its rows.

3.2.2 Local training

Node i trains with the detection loss in which the classification term of every unowned class is masked:

$$\mathcal{L}_{\text{cls}}^{(i)} = \sum_{c\in\mathcal{C}} [c \in \mathcal{C}_i]\ \text{BCE}(\hat{p}_c, y_c). \tag{3}$$

An unlabelled object of another class therefore produces no gradient that teaches the model it is background. Local training uses SGD (momentum 0.9, weight decay

$5 \cdot 10^{-4}$, batch 4, 2 local epochs per round or update). The learning rate decays linearly from $8 \cdot 10^{-4}$ to 1% of it, driven by the progress of the whole federation. A node's momentum stays on its own machine.

3.2.3 Aggregation schemes

Figure 2 illustrates the three schemes, and Figure 3 works through one update, entry by entry. We name them after the methods they follow, FedAvg [1], cyclic weight transfer (CWT) [10] and FedAsync [11], and use these names throughout; in COFY, all three take each class's rows only from its owner.

- *FedAvg, parallel synchronous.* In each round all nodes train from the same global model. The server waits for all of them, averages the shared parameters weighted by data size, and takes each class row from its owner (R3).
- *CWT, sequential synchronous.* Nodes train one at a time, in an order the server draws at random for every cycle. Each node starts from the model left by the previous one, and the server applies its update in full. This is synchronous: the server schedules the turns and a cycle needs every node. The final model averages the last four global versions (stochastic weight averaging [12]), which removes the bias towards the last node.
- *FedAsync, concurrent asynchronous.* Nodes train continuously and independently, as fast as their hardware allows. An update computed from version v_{pull} arrives when the global model is at version v_{now}, with staleness $s = v_{\text{now}} - v_{\text{pull}}$. It is applied at once:

$$\theta_{\text{shared}} \leftarrow \theta_{\text{shared}} + w\,\Delta_{\text{shared}}, \qquad w = \frac{1}{1+\alpha s},\ \alpha = 0.5, \tag{4}$$

$$\theta^{(c)} \leftarrow \theta^{(c)} + w\,\Delta^{(c)} \qquad \text{only for } c \in \mathcal{C}_i, \tag{5}$$

  where Δ is the node's change from the model it pulled [11]. BatchNorm running variances are clamped at zero, because stale deltas can push them negative. The final model averages, over nodes, the global version right after each node's last update. Arrival order thus comes from timing alone, never from a server-side choice.

3.2.4 Calibration

Merging moves the shared features, while each class row was fitted to the features its owner saw. A short *calibration* re-aligns them without breaking R1–R3. Each node passes its own training images once through the final model, without changing any weight, and records for every BatchNorm channel the mean μ_i and variance σ_i^2 as BatchNorm itself estimates them, as cumulative averages over its batches [60], together with its number of batches b_i. The server pools first and second moments:

$$\mu = \frac{\sum_i b_i\,\mu_i}{\sum_i b_i}, \qquad \sigma^2 = \frac{\sum_i b_i\,(\sigma_i^2 + \mu_i^2)}{\sum_i b_i} - \mu^2 = \frac{\sum_i b_i\,\sigma_i^2}{\sum_i b_i} + \frac{\sum_i b_i\,(\mu_i - \mu)^2}{\sum_i b_i}. \tag{6}$$

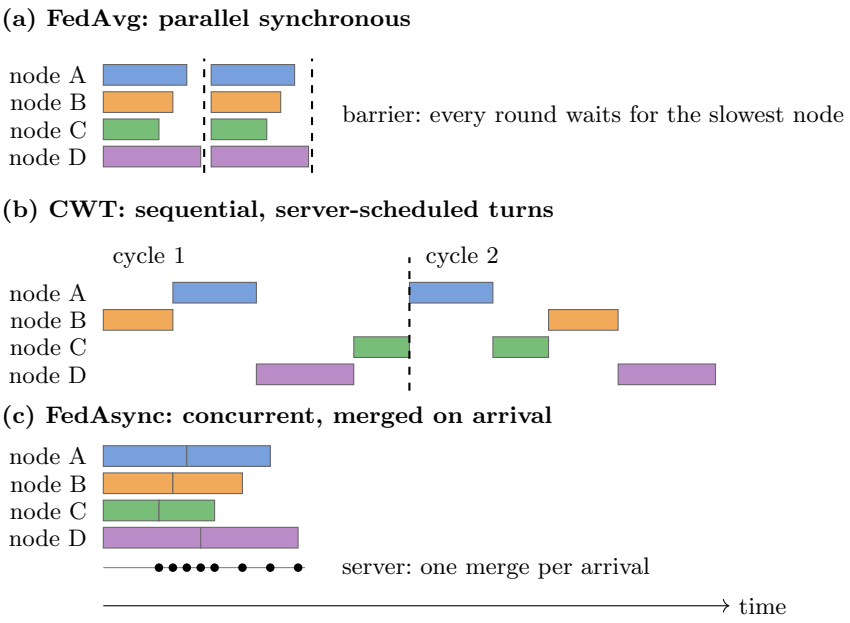


Fig. 2 The three schemes with four nodes of different speeds, two cycles each. In (a), nodes train in parallel and the server averages after a barrier. In (b), one node trains at a time, starting from the previous node's model, in a new random order every cycle. In (c), nodes train concurrently and each update is merged on arrival. Every scheme trains the same number of node-epochs. CWT needs about three times the wall-clock time (Section 5.2).

The pooled variance therefore includes the spread of the node means around the pooled mean, as the variance of the pooled data does; a weighted average of the node variances alone would omit it. Each node then re-fits only its own class rows on the frozen model with the pooled statistics, for one epoch, and the server replaces those rows (Figure 3c).

The synchronous form (*closing calibration*) needs a barrier: all statistics, then all rows. We designed a *progressive calibration* for the asynchronous scheme, so that nobody waits. The final model is published at once. Every contribution, as it arrives, makes the server release an updated model. Nodes not yet heard from enter Eq. (6) with the final model's own statistics, weighted by their batch counts, which are known from their training uploads. Once every node has contributed, each re-fits its rows once more on the complete pool. Nodes can download a usable model at any time, and the calibration is complete when every node has been seen. To keep the comparison fair, the centralized reference receives the same calibration.

3.3 Deployment and defence against malicious nodes

The same schemes run in a secured client–server implementation, and the server can screen every upload against a malicious node.

(a) Merging one upload: node C under FedAsync

		global model v_7	node C's change Δ	taken by the server?	applied $w\,\Delta$	global model v_8
	shared weight	0.820	+0.060	yes	+0.030	**0.850**
	BatchNorm mean	0.410	−0.040	yes	−0.020	**0.390**
D	basophil	0.310	≈0	no: owner D	0	**0.310**
B	eosinophil	0.270	≈0	no: owner B	0	**0.270**
C	erythroblast	0.120	+0.080	yes: owner	+0.040	**0.160**
C	immunoglobulin	0.220	+0.050	yes: owner	+0.025	**0.245**
D	lymphocyte	0.350	≈0	no: owner D	0	**0.350**
D	monocyte	0.180	≈0	no: owner D	0	**0.180**
A	neutrophil	0.290	≈0	no: owner A	0	**0.290**

Staleness. Node C pulled v_5 and its change arrives at v_7: $s = 2$, so $w = 1/(1 + 0.5\,s) = 0.5$.

Grey parts are taken from every node; **coloured rows** only from the class's owner (R3).

Before the merge, the defence may scale each part down (Algorithm 1).

(b) The same entries under the other schemes

CWT. Node C trains from the latest model, so $s = 0$ and $w = 1$: its change is applied in full. Shared weight 0.880, BatchNorm mean 0.370, erythroblast 0.200, immunoglobulin 0.270; the other five rows are unchanged. The next node in the turn order starts from this model.

FedAvg. All four nodes train from the shared weight 0.820 and reach A 0.860, B 0.800, C 0.880, D 0.840, on 255, 235, 289 and 289 images: data-weighted mean 0.847. With the defence, trimmed mean of the changes, the lowest and highest dropped: $0.820 + 0.030 = 0.850$. Each row takes its owner's value, with no averaging.

(c) Calibration, once, after the last update

Step 1: pool BatchNorm statistics

node	mean μ_i	variance σ_i^2	batches b_i
A	0.20	1.00	64
B	0.35	0.90	59
C	0.50	1.20	73
D	0.30	0.80	73

For one channel, the server pools them with Eq. (6), $\sum_i b_i = 269$:

$\mu = 0.341$

$\sigma^2 = 0.978 + 0.012 = 0.990$

the mean of the node variances plus the spread of the node means. The pooled values replace the statistics of every BatchNorm channel.

Step 2: owners re-fit their own rows

A → rows of neutrophil

B → rows of eosinophil

C → rows of erythroblast, immunoglobulin

D → rows of basophil, lymphocyte, monocyte

With every other weight frozen and the pooled statistics in place, each owner trains its rows for one epoch on its own images; the server writes each row back from its owner, with no averaging.

Progressive form (FedAsync): the pool grows as statistics arrive, nodes not yet heard from enter with the final model's statistics and their batch counts, a model is released after every arrival, and every owner re-fits its rows once more on the complete pool.

Fig. 3 Merging and calibration in COFY, a worked example with the classes and nodes of the WBC use case (colours as in Figure 1). One entry of each part is shown; every entry of a part follows the same rule, and a class row stands for its weights and bias at every scale. (a) One upload of node C merged by FedAsync: shared weights and BatchNorm statistics are taken from every node, scaled by the staleness weight w, while a class row is taken only from the class's owner. The rows of classes that C does not own barely move in its training, since their classification loss is masked and only weight decay acts on them, and the server ignores them. (b) The same entries under CWT and FedAvg, with and without the defence. (c) The calibration after the last update: BatchNorm statistics pooled by Eq. (6), then each owner re-fits its own rows on the frozen model. The values are illustrative; the rules, the staleness weight and the batch counts are those of the study.

3.3.1 Networked implementation and security

The schemes evaluated here run unchanged in a client–server implementation. Nodes are client processes, one per site, and the server manages "rooms", one per federation. Security measures follow the threats each addresses:

- *Authentication.* A room password plus a per-node secret token, stored hashed, is required on every request. Administrative actions are allowed only locally or with an admin token.
- *Safe payloads.* Only plain tensors are exchanged, deserialized without code execution. Every upload is checked against the room's model: same tensor names and shapes, finite values, a size limit, and an idempotent upload identifier.
- *Ownership.* The server takes class rows only for the classes in its own registry, never for the classes an upload claims (R3).
- *Transport.* Optional HTTPS, with private certificate authorities supported.
- *Availability.* Heartbeats mark lost nodes; their turns are skipped in CWT. Requests are retried with back-off, and state is persisted for restarts.

Server-side evaluation exists only in a developer mode, because in deployment the server holds no data (R2). CWT is enforced by the server: a node asking out of turn is told to wait. Both schemes and both calibration forms were exercised end to end with a live server and separate client processes on one machine; the code release includes this demonstration for all three use cases. The implementation has not been run across sites, and no result in this article comes from it: every reported run uses the simulation of Section 4.1.

3.3.2 Ownership-aware robust aggregation

Byzantine-robust rules compare simultaneous updates and keep the majority view [38, 39]. Under disjoint ownership this fails in three ways:

- a class row has a single contributor, so no majority exists;
- honest updates of shared layers differ by design, because each node optimizes a different objective;
- in CWT and FedAsync the server sees one update at a time.

Defences that use server data are excluded by R2. We therefore bound what each update can do, part by part, against references the attacker cannot set alone (Algorithm 1):

- Ownership is enforced from the registry.
- Each part of an update is scaled down if its size, the Euclidean norm of its change from the model it was trained from, exceeds $\kappa = 2$ times a reference. The parts are the shared weights, the BatchNorm statistics, and each owned class row (its weights and bias at every scale).
- For each type of part, the server keeps the sizes of the 16 most recent parts it received (four per node), as sent and before any scaling, each with its sender. The reference is the median, over the other nodes in that list, of each node's median size. A class row has no peer for its class, so the list for rows pools the rows of every class: each

Algorithm 1 Ownership-aware screening of one upload from node i

Require: upload θ_i, model it was trained from θ^{pull}, registry $\mathcal{C}_i$; for each part type $t \in \{\text{shared}, \text{BatchNorm}, \text{row}\}$ a list H_t of the 16 most recent entries (node, size), initially empty

$\Delta \leftarrow \theta_i - \theta^{\text{pull}}$; owned classes $\leftarrow \mathcal{C}_i$ ▷ never the classes the upload claims

for each part $p \in \{\text{shared}, \text{BatchNorm}\} \cup \{\text{row } c : c \in \mathcal{C}_i\}$, of type t **do**

  $J \leftarrow$ nodes $j \neq i$ with an entry in H_t; $m_j \leftarrow \text{median}\{x : (j, x) \in H_t\}$

  $b_p \leftarrow \kappa \cdot \text{median}_{j \in J}\, m_j$ **if** $|J| \geq 3$ **else** ∞

  $n_p \leftarrow \|\Delta_p\|$; $\Delta_p \leftarrow \Delta_p \cdot \min(1,\ b_p / n_p)$; log (i, p, n_p, b_p)

  if $t \neq$ row: append (i, n_p) to H_t

end for

append (i, n_p) to H_{row} for every owned row p ▷ after the loop

return $\theta^{\text{pull}} + \Delta$ to the scheme's merge rule

upload adds one entry per owned row, after all its rows have been screened, so that rows of one upload never bound each other.

- The lists start empty. Until a list holds entries from at least three other nodes, the part is accepted as sent. With one attacker among four, the median of three nodes' medians lies within the range of the honest ones.
- In FedAvg rounds, the shared deltas are combined by a coordinate-wise trimmed mean instead of the data-weighted mean, because reported data sizes are not trusted.

Every decision is logged for audit. The bounds limit the size of an update, not its direction; Section 5.4 shows the consequence.

3.4 Use cases

Table 1 lists the three use cases, and Table 2 their classes, owners and instances:

- *KFM12*: twelve classes of parasite eggs in animal faecal samples, imaged with the Kubic FLOTAC Microscope [32], the device for which the KFM detectors were developed [33, 34]. The 15,448 images were acquired between January 2023 and March 2025 from faecal samples of livestock, mainly sheep, cattle and goats, collected for routine parasitological diagnosis by the CREMOPAR research group of the Department of Veterinary Medicine and Animal Production, University of Naples Federico II. The dataset is released with this article (Declarations);
- *WBC*: seven white-blood-cell classes, with bounding boxes [61] on the PBC dataset [62];
- *DENTEX*: the 32 tooth classes of the enumeration subset [63]. It joined the study after KFM12 and WBC, with its configurations frozen and hashed before its first run (Section 4.2).

The three use cases were chosen to cover the conditions a class-owned federation can meet (Table 1, Figure 4). Classes share an image never in WBC, sometimes in KFM12 and always in DENTEX. The classes are strongly imbalanced in KFM12, mildly in WBC and nearly uniform in DENTEX. Two use cases are microscopy, with 12 and 7 classes, and one is radiography, with 32.

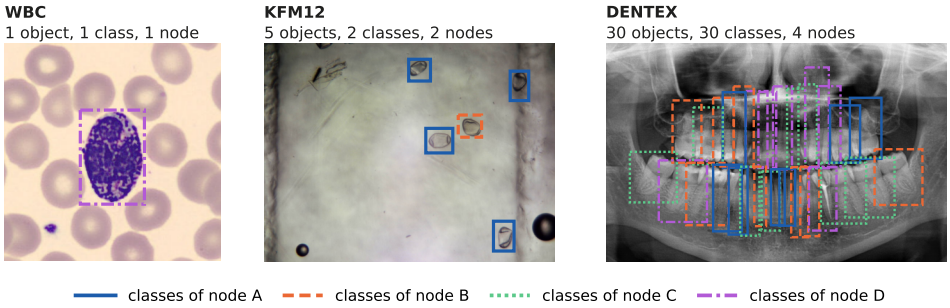


Fig. 4 One evaluation image of each use case with every labelled object, coloured by the node that owns its class (node colours as in Figures 1 and 2). WBC: one cell, one class. KFM12: a few eggs, of the species of one or two nodes. DENTEX: every tooth is a class, and every node's classes appear in every radiograph. Images are chosen by a fixed rule (`tools/make_usecase_figure.py`).

Each dataset is split once, at image level, into training images (80%) and an evaluation set (20%). Classes are assigned greedily to four nodes so that labelled instances are balanced. Every training image goes to each node owning a class present in it, labelled only for that node's classes. Every scheme and the centralized reference train on the same images. For KFM12 and WBC these are a class-stratified tenth of each node's training images, drawn with the seed: every class keeps a tenth (at least one) of the node's images that contain it, so that the three use cases train on a comparable number of images. DENTEX has only 508 training images and trains on all of them. The other training images are not used. Every class count in this article (Tables 1 and 2, Section 5.2) is a count of what was trained on: the labelled instances in the images of these subsets, as a mean over the seeds. Table 2 also gives the counts of the full training split, from which the subsets are drawn, about ten times larger. The evaluation set is fully labelled, deduplicated and never subsampled, and every model is scored on it.

A WBC image is a crop around a single cell: one object of one class, so no node ever sees an object of another node's class. A KFM12 image holds 2.5 eggs on average, usually of one species (1.1 classes per image), and about a fifth of a node's images also contain eggs of species owned by other nodes. A DENTEX panoramic radiograph shows 29 teeth on average, each of them a class of its own, so every image contains classes of all four nodes: every node holds every image, with only its own eight teeth labelled and the other 24 unlabelled. This is the extreme of disjoint ownership. The masked loss (Section 3.2) keeps those unlabelled teeth from being learned as background, so nothing has to be hidden in the images, and the centralized reference trains on the same partial labels, so *Fed:Cent* separates the cost of federation from the cost of partial labels. The class frequencies differ as well (Table 2). KFM12 is strongly imbalanced: in the images trained on, its commonest class has 669 labelled eggs per seed on average and its rarest about 5 (4–7). WBC is mildly imbalanced (75–255 cells per class), and DENTEX is nearly uniform (262–506 per tooth, the third molars being the rarest).

Table 1 Use cases. Train: unique images every scheme and the centralized reference train on, per seed (KFM12: 1,385–1,400 over seeds 100–109). Eval: the fully labelled evaluation set every model is scored on. Objects and classes per image: means over the evaluation set. Unlabelled objects: share of a node's training images that also contain objects of classes owned by other nodes, which that node sees without labels (mean over nodes). Imbalance: labelled instances trained on of the commonest class over those of the rarest. Budget: training epochs, set from where a centralized pilot run's mAP50 plateaus (Section 4.1).

Use case	Images	Classes	Train	Eval	Objects per image	Classes per image	Unlabelled objects	Imbalance	Budget
KFM12	microscopy, parasite eggs	12	1,391	3,089	2.5	1.1	23%	142×	30
WBC	microscopy, blood cells	7	1,068	2,668	1.0	1.0	0%	3.4×	20
DENTEX	panoramic X-ray, teeth	32	508	126	28.7	28.7	≈100%	1.9×	20

Table 2 Classes, their owner node and their labelled instances. Train: in the images trained on, per seed (mean over seeds 100–109; KFM12 and WBC train on a class-stratified tenth of their training images, DENTEX on all of them); every count in this article refers to these. Split: in the full training split from which the subsets are drawn. Eval: in the evaluation set. KFM12 class names as labelled in the dataset.

KFM12					*WBC*				
Class	Owner	Train	Split	Eval	Class	Owner	Train	Split	Eval
strongili	C	620	5,815	1,350	basophil	D	88	877	196
fasciola	B	611	6,110	1,553	eosinophil	B	235	2,354	579
eimeria intricata	A	5	50	16	erythroblast	C	75	746	176
calicophoron	A	669	7,070	2,034	immunoglobulin	C	214	2,139	553
nematodirus	C	132	1,232	353	lymphocyte	D	93	928	233
dicrocoelium	B	22	201	58	monocyte	D	108	1,079	279
metastrongili	B	32	307	55	neutrophil	A	255	2,553	652
strongyloides	C	106	652	187					
moniezia benedeni	A	34	342	90					
ascaris	D	533	5,671	1,248					
ancylostoma	D	251	2,508	627					
trichuris trichiuria	B	120	1,198	265					

DENTEX: 32 teeth in FDI numbering, eight per node; train and eval: range over a node's eight teeth.

Owner	Teeth	Train	Eval
A	14, 26, 27, 28, 31, 32, 44, 45	282–505	85–125
B	13, 15, 17, 33, 34, 38, 42, 46	312–506	82–124
C	16, 22, 23, 36, 37, 41, 43, 48	319–506	83–124
D	11, 12, 18, 21, 24, 25, 35, 47	262–503	79–125

4 Experimental design

All arms share one training protocol (Section 4.1). The two schemes are run once, on ten seeds, in a non-inferiority study whose code and analysis were frozen before its first run (Section 4.2). Secondary analyses describe who bears the cost of federation and what it costs to run, and compare the schemes with standard methods and two

baselines (Section 4.3). A separate study measures robustness to a malicious node (Section 4.4).

Generative AI was used in this work as follows. The authors designed the system architecture, the study and its experiments. Claude Opus 5.5 (Anthropic) was used to draft the manuscript, to develop part of the code, and to automate the experimental runs. The code it developed implements components conceived and designed by the authors, who also validated it. The authors reviewed and verified all content and take full responsibility for it. No AI-generated images are included.

4.1 Training protocol

All arms use the same recipe: 2 local epochs per round or update, and linear learning-rate decay to 1% over the whole budget. The budget comes from a centralized pilot run of up to 100 epochs, scored every 5 epochs. It is the peak of the three-point moving average of the pilot's mAP50: 30 epochs for KFM12. WBC and DENTEX plateau by the first checkpoint (epoch 5) and are trained for 20 epochs. From epoch 5 on, their pilot curves stay within 0.045 and 0.02 mAP50 of their best, with no trend. The budgets correspond to 15, 10 and 10 rounds or updates per node on KFM12, WBC and DENTEX. With a constant learning rate, the score of a single run swings by ± 0.02 mAP50 between checkpoints five epochs apart, as much as the effects being measured; decaying the rate halves the seed-to-seed spread of the centralized model.

The centralized reference and every federated scheme see the same examples, as often (Table 3). An example is an image with one node's partial labels and class mask, and a node's examples are the images of its training subset. An image that contains classes of several nodes is one example per owner. The centralized model passes through each node's examples in turn, with that node's mask, so it processes such an image as often as the federation does: once per owner and epoch, which on DENTEX means four times. Both draw batches of four from one node's examples at a time, so they take the same number of optimizer steps, and in both the learning rate decays linearly with each node's progress through its own epochs. Three differences remain:

- each node keeps its own SGD momentum (0.9) across its rounds or turns, while the centralized model keeps one;
- the learning rate warms up linearly over 20 steps at the start of every federated round or turn, but only once in centralized training, so the federated nodes train with 3.7–6.9% less summed learning rate;
- the centralized model visits the nodes' examples in a fixed order, one epoch at a time, while CWT visits them in turns of two epochs in a random order, and FedAvg and FedAsync train copies in parallel.

These differences are part of what *Fed:Cent* measures. The first and third are inherent to federated training; the second is an implementation choice.

Concurrency is simulated on one GPU with a seeded discrete-event clock:

- each node draws a machine speed $1 \pm U(0, 0.3)$ per seed;
- an update lasts (images $\times$ epochs) / speed, $\times(1 \pm U(0, 0.2))$ jitter;

Table 3 Training exposure per seed, mean over seeds 100–109. The first five columns are the same for the centralized reference and every federated scheme. Examples: an image with one node's partial labels; an image with classes of several nodes is one example per owner. Steps: optimizer steps of four examples. Learning rate: the summed learning rate of the federated nodes relative to the centralized model, lowered by the warm-up that restarts at every round or turn.

Use case	Images	Examples per epoch	Steps per epoch	Epochs	Steps	Learning rate
KFM12	1,391	1,407	354	30	10,611	−5.3%
WBC	1,068	1,068	269	20	5,380	−6.9%
DENTEX	508	2,031	508	20	10,160	−3.7%

- updates are applied in the order in which they finish on that clock;
- each training is executed when its pull happens, so staleness and arrival order are exactly those of the timeline (average staleness 2.4–2.7).

The clock models computation only: no transfer time, no failures and no node that leaves. It sets the order and staleness of the updates; the waiting times reported in Section 5.2 are derived from it, not measured.

4.2 Non-inferiority study

The two schemes were run once, on ten seeds, with their code, configurations and analysis frozen beforehand:

- *Schemes:* CWT with closing calibration, and FedAsync with progressive calibration and refresh.
- *Seeds:* ten seeds (100–109), not used to design or tune the schemes. Each seed also draws the node order of CWT, the machine speeds of FedAsync and the training subsets.
- *No early stopping and no added seeds.*
- *Frozen artefacts:* the analysis script, configurations and code were frozen in a separate copy and recorded by SHA-256 on 28 October 2026 at 10:54 CET, before the first run started, and the study runs from that copy. The DENTEX configurations and label fingerprints were frozen and recorded in the same way on 1 November 2026, before its first run.
- *What is stated:* for each scheme, the highest share of centralized mAP50, in steps of 5 percentage points, that its lower bound (Eq. (2)) exceeds on every use case, and the same level for each use case.
- *Evidence:* the evaluation protocol, the SHA-256 manifests, the frozen code and analysis script, and the run-level records of every run reported are archived with the code release (Declarations). Appendix A.4 lists the seed-level values.

Each level is a confidence statement: with 95% two-sided confidence on each use case, the scheme keeps at least that share of centralized mAP50 there. No clinical threshold exists for this endpoint, so we state the level reached rather than test one acceptable loss. A reader who would accept a given loss can compare it with the lower bounds, or with the seed-level values of Appendix A.4.

4.3 Secondary analyses and baselines

- *Fairness:* per-class retention (federated AP50 / centralized AP50 of the class, means over seeds), the worst class, the number of classes below 85%, retention per node, and its Spearman correlation with the number of training boxes of the class (permutation p-values).
- *Cost:* measured on the single GPU of the simulation: GPU time, and GPU energy from the board's power draw. Computed: the communication volume, from the model size and the number of transfers. Derived from the timing model: the deployment wall-clock. None of these was measured on a multi-site deployment.
- *Standard methods:* FedProx [16], FedBuff [36] and CA2FL [37], under the same protocol and post-processing, on seeds 0–2 of KFM12 and WBC. They change only the rule that combines updates, so they run unchanged under class ownership. Methods for partially labelled federations cannot be compared directly. They address classification or segmentation, with losses built for those tasks [8, 9, 20–23], or they train on the server [13, 20], which R2 excludes, or they keep BatchNorm local [19], which leaves no single global detector. Federated detectors assume a shared label set [25, 27]: without the masked loss, every node would learn the other nodes' unlabelled objects as background.
- *Local-only baseline (added after the main results):* each node trains a detector alone on its own data and classes, with the same recipe and the same number of epochs it trains in the federation, and each class is scored on its owner's model, as if every class were reported by its owner. It measures what federation adds over not federating. Seeds 100–102 on all three use cases.
- *Centralized reference with weight averaging (added after the main results):* on WBC, the centralized model trained with the averaging of CWT (the mean of four snapshots over the last two epochs, one every half epoch, as one sequential turn trains on a quarter of the data for two epochs), then calibrated. Seeds 100–109. It tests whether averaging explains why the federated schemes score above centralized on WBC (Section 5.1).

4.4 Robustness study

Table 4 lists the attacks, and Figure 5 shows them on images of the three use cases. The study runs on WBC, the use case with the shortest runs, so that its 120 runs fit the compute budget. With seeds 0–2 and the attacker rotating with the seed, each attack is run on each scheme with and without the defence, and the defence is also run on clean federations to measure its cost. Models are scored as the server evaluates them after the last update, that is, the last global version, without calibration. We report mAP50 and *Fed:Cent*, mAP50 on the attacker's classes and on everyone else's, and the share of uploads the defence scaled, for the attacker and for honest nodes. For the backdoor we also report the attack success rate: the share of triggered evaluation images on which the target class is detected on the trigger, at confidence ≥ 0.25 and IoU ≥ 0.5.

Standard evaluation keeps at most 300 detections per image after non-maximum suppression, shared by all classes. A class whose scores are high everywhere can fill that

budget and leave none to the others. To measure this effect, the defended sign-flip runs and the FedAsync BatchNorm-poisoning seed that collapsed were repeated with their final model kept, and these models and the defended clean runs are also scored with a *per-class budget*, in which every class keeps its own 300 detections. This evaluation was added after the sign-flip results. When no class fills the shared budget, the two evaluations give identical results. The per-class budget is a containment measure at inference, not a defence. It leaves the model as it is: a compromised class still fires everywhere and its own detections stay useless, but it can no longer take the other classes' share of the output. To help, it must be in place before an attack is known. It also has an operational cost: up to 300 candidate detections per class and image instead of 300 in total, which lengthens post-processing and enlarges the output that downstream software receives.

After these results, we tested four candidate defences against the attacks the defence did not stop. All are post hoc: they were designed after seeing the robustness results and run on the same use case and seeds, so their results are limited demonstrations, not established performance. All keep the rules of Section 3.1: the server holds no data, and only a class's owner changes its rows.

- *Median pooling of BatchNorm statistics in the calibration.* The calibration discards the BatchNorm statistics accumulated in training and pools those the nodes compute on their own images (Section 3.2.4). The study's rule weights each node's statistics by its batch count (Eq. (6)). The candidate takes their coordinate-wise median [39], which one node among four cannot move outside the range of the honest ones. Unlike class rows, BatchNorm statistics come from every node, so they can be compared; corrupted statistics are known to break federated training [64]. We apply the BatchNorm step alone to the final models of the BatchNorm-poisoning runs, repeated with their final model kept, and of the defended clean runs. Every node is honest in the calibration, or the attacker also poisons the statistics it sends, with the corruption of Table 4.
- *Screening inputs at the nodes.* Detector Cleanse [48] checks a detection at inference by blending the detected region with crops of other objects. A trigger keeps its class whatever is blended in, so the entropy of the class scores stays abnormally low. A node blends each detection with 32 of 64 crops of its own labelled objects (weight 0.5) and normalizes the sigmoid class scores before computing the entropy. It checks the detections of classes it does not own, and flags those whose average entropy falls below the 5th percentile of the entropies of the detections on its own validation images. This threshold was fixed after two pilot rules, on 10–20 images of seeds 0 and 1, flagged too many clean images. The screening node is the honest node after the attacker. We measure the attack success rate on 100 triggered evaluation images with and without screening, the share of the same images without the trigger that are flagged (false alarms), and the AUROC of the entropy between the backdoor's detections and the checked detections on clean images.
- *Bounding consecutive merges jointly in FedAsync.* The updates a node merges in a row, with no other node's update in between, are bounded together for the shared weights and the BatchNorm statistics: their cumulative change must stay within the bound of one update. Per-update bounds cannot stop a direction that persists

Table 4 Attacks. One node of four is the attacker. Data attacks change the attacker's training data; upload attacks change what it sends.

Attack	What the attacker does	Detection-specific
noise	sends Gaussian noise, 3× its honest update's size, tensor by tensor (faulty node)	no
sign flip	sends the opposite of its honest update, same size	no
scaling	boosts its update ×10 (model replacement [43])	no
weight inflation	reports 100× its image count (FedAvg weights)	no
BatchNorm poisoning	shifts running means by 3σ, scales variances ×9	yes: statistics shared by every class
row hijacking	claims every class, makes others' classes fire everywhere	yes: per-class rows
box corruption	drops half its boxes, moves and resizes the rest	yes
phantom-object backdoor	checkerboard trigger in 30% of its images, labelled as its class [48]	yes

when no honest update follows, as in the concurrent seed whose attacker sent its last three uploads in a row. We run it on the FedAsync BatchNorm-poisoning runs and on clean FedAsync runs, with the defence, seeds 0–2.

- *Peer validation with per-class rollback.* After every new global version, each node runs the model on 32 of its own training images, without labels, and reports one number per class: the share of the head's locations where the class scores at least 0.25. A node votes against a class that covers at least 2% of the locations on its images and at least ten times the median of the other classes there; its owner does not vote on it. A class flagged by a majority of the nodes that do not own it at two checks in a row is rolled back to its rows at the last check it passed, and its owner's later row updates are discarded [45]. Its rows thus only ever hold values its owner produced (R3): the other nodes can veto them, never write them. The node named as attacker also lies in its reports, claiming that every class it does not own covers every location. The thresholds were set on the kept final models of the robustness study before any of these runs: clean and backdoored classes cover at most 0.1% of the locations, the sign-flipped class 16–90%. We run it on the sign-flip runs with the defence and on clean runs, in the three schemes, seeds 0–2. A variant also excludes the owner of a rolled-back class: from then on the server discards its uploads on arrival, shared layers and BatchNorm statistics included, and its reports no longer count in the vote. Until a rollback it behaves exactly as peer validation alone, so the clean runs above serve for both. We run it on the defended sign-flip runs, in the three schemes, seeds 0–2.

5 Results

We first report the cost of federation on the three use cases (Section 5.1). We then show who bears that cost and what each scheme costs to run (Section 5.2), compare the schemes with standard methods and a local-only baseline (Section 5.3), and report the robustness study (Section 5.4).

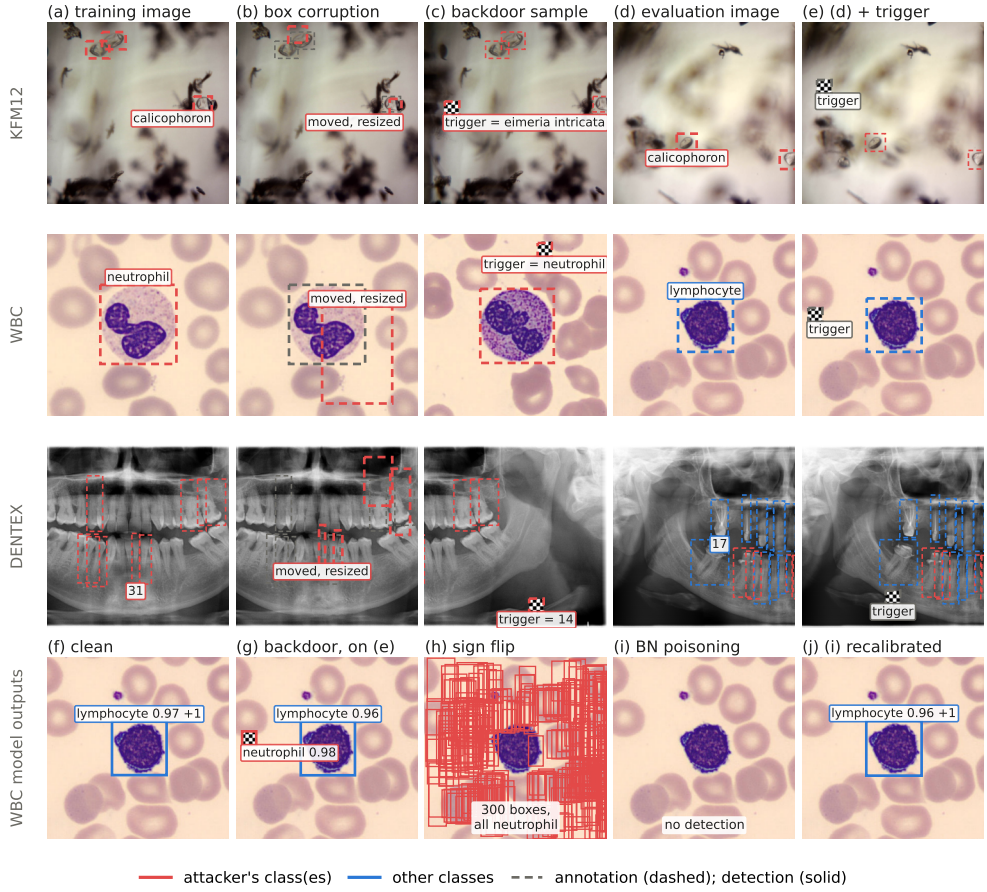


Fig. 5 Attacks of the robustness study. Rows 1–3: the data attacks on a KFM12, a WBC and a DENTEX image, with node A, the attacker of seed 0, whose classes are in red. (a) One of its training images with its boxes; (b) the same image with corrupted labels: half the boxes dropped (grey only), the others moved and resized; (c) a backdoor sample, with the trigger labelled as the attacker's first class; (d) an evaluation image without that class and (e) the same image with the trigger. The WBC panels come from the study's attack data; KFM12 and DENTEX, on which the robustness study was not run, apply the same rules. Panels are square crops as wide as the image's short side (for DENTEX, the central part of the radiograph). Row 4: what federations trained under attack detect. The robustness study trained them on WBC only, so these outputs exist only for WBC: detections (confidence ≥ 0.25) of final models trained with the defence: (f) clean federation on (d); (g) backdoored federation on (e); (h) sign flip on (d), where the attacker's class fills the budget of 300 detections; (i) BatchNorm poisoning in the FedAsync seed that collapsed (attacker node C), final model before calibration, on (d); (j) the model of (i) after the BatchNorm step of the calibration with median pooling, the attacker also poisoning the statistics it sends. Each object shows its most confident detection; $+k$ counts the other classes detected on it at lower confidence, as models trained on partial labels do. Images are chosen by a fixed rule (first match in file order).

Table 5 Seeds 100–109, not used to design the schemes; frozen code, configurations and analysis. *Fed:Cent*: federated / centralized mAP50 ×100, paired by seed, both after the same post-processing; mean, standard deviation and two-sided 95% t-interval. Loss: centralized minus federated mAP50, mean with its upper 95% bound. mAP50-95: the same ratio for mAP50-95, with its lower bound.

Scheme	Use case	n	*Fed:Cent* (sd)	95% CI	Loss (upper)	mAP50-95 (low)
CWT	KFM12	10	98.6 (2.7)	96.7–100.6	+0.010 (+0.025)	98.6 (96.0)
closing calib.	WBC	10	105.4 (9.4)	98.7–112.1	−0.025 (+0.007)	105.3 (98.7)
	DENTEX	10	98.8 (3.7)	96.2–101.4	+0.012 (+0.037)	98.6 (95.6)
FedAsync	KFM12	10	93.6 (2.0)	92.1–95.1	+0.047 (+0.058)	93.2 (91.4)
progressive	WBC	10	130.8 (12.3)	122.0–139.6	−0.152 (−0.114)	129.8 (121.1)
calib. + refresh	DENTEX	10	99.7 (0.5)	99.4–100.1	+0.003 (+0.006)	101.4 (100.6)
Centralized mAP50 (calibrated)			KFM12 0.734, WBC 0.502, DENTEX 0.959			

5.1 The cost of federation

Table 5 gives the results on KFM12, WBC and DENTEX, computed once on all ten seeds:

- *CWT* keeps at least 95% of centralized mAP50 on every use case. Its lower bounds are 96.7% on KFM12, 98.7% on WBC and 96.2% on DENTEX.
- *FedAsync* keeps at least 90% on every use case. Its lower bounds are 92.1% on KFM12, 122.0% on WBC and 99.4% on DENTEX: use case by use case, at least 90% on KFM12 and 95% on DENTEX, and on WBC above centralized even at its lower bound.

With ten seeds, the t-interval rests on an approximate normality of the mean ratio. As a check (Appendix A.4), four other lower bounds were computed: a t-interval of the log ratio, a bias-corrected and accelerated bootstrap, the exact Wilcoxon interval of the Hodges–Lehmann estimate, and the t-interval with any one seed left out. For FedAsync all of them stay above 90% (lowest 91.8%, KFM12, one seed left out). For CWT all stay above 95% except two on DENTEX, the bootstrap (94.4%) and the Wilcoxon interval (94.7%), both sensitive to the skew that the one weak seed, 106, gives the ten values. CWT's 95% on DENTEX therefore rests on the t-interval; under every interval it keeps at least 90%.

Three features of the results on KFM12 and WBC deserve comment:

- *Spread.* The seed-to-seed spread is small on KFM12 (sd 2.0–2.7 points), so the lower bounds are close to the means. This is the effect of the protocol's learning-rate decay (Section 4.1).
- *The cost of concurrency on KFM12.* FedAsync costs 0.047 mAP50 (upper bound 0.058), about five times the cost of CWT. Section 5.2 shows that this cost falls on rare classes.
- *WBC above centralized.* Both federated schemes score above centralized on WBC, FedAsync by about a third.

On the last point: WBC saturates earliest. Its centralized pilot does not improve after the first checkpoint (epoch 5), so most of the budget is spent on a plateau. Federated

training averages model versions, over the last turns or over nodes, and averaging weights along such a plateau is known to improve generalization [12]. We tested this explanation by giving the centralized model the averaging of CWT (Section 4.3, seeds 100–109). Averaging raised its calibrated mAP50 from 0.502 to 0.516, to 102.9% of the plain centralized model (95% CI 99.2–106.6). Against this averaged reference:

- *CWT:* 102.5% (95% CI 96.2–108.9). Its excess over centralized is within what averaging explains. Before calibration it is below centralized (98.3%).
- *FedAsync:* 127.1% (95% CI 120.0–134.3). Its excess is already present before calibration (126.0%), and averaging over time does not explain it. FedAsync also averages across nodes, over each node's latest version, which a single centralized model cannot reproduce; we did not test whether this is the cause.

We do not interpret values above 100% as a benefit of federation.

DENTEX tests the system where class ownership is most entangled. Every radiograph is held by all four nodes, each with only its own eight teeth labelled. A node therefore sees the other 24 teeth in every image as unlabelled. The centralized pilot learns the task quickly: mAP50 0.945 after 5 epochs and 0.958 at best (epoch 15). Most of the room for a federation loss is therefore in how the shared features are merged, not in learning each class.

Both schemes stay close to centralized on DENTEX (Table 5): CWT at 98.8% (lower bound 96.2%), FedAsync at 99.7% (99.4%). The worst class is at 98% under CWT and 97% under FedAsync, and every node keeps 99–100% (Table 6). Two points qualify this result:

- *One weak seed.* The CWT interval is wider because of seed 106, on which the CWT model reached 88.4%, with every class lower rather than a few. The run completed normally and counts like any other seed. The other nine seeds are within 1% of centralized.
- *Near the ceiling.* With centralized mAP50 at 0.959, little room is left for a loss. DENTEX tests the extreme of entangled ownership on an easy task, not on a hard one.

5.2 Fairness and cost

Mean ratios do not say who bears the cost of federation, nor what each scheme costs to run; we report both.

5.2.1 Fairness across classes and nodes

Mean ratios can hide large losses on single classes (Table 6). The CWT and FedAsync rows use seeds 100–109. FedAvg was not part of this study, so its rows use seeds 0–2, on which CWT and FedAsync gave the same picture (Table 7).

- *CWT.* It keeps almost every class close to centralized. Its worst class on WBC is at 94%. On KFM12 only one class in twelve falls below 85%: eimeria, the rarest, with about 5 training boxes per seed (4–7), at 75%. Its paired difference has a confidence interval that includes zero (-0.107, 95% CI -0.242 to $+0.028$); with 16 instances

Table 6 Per-class retention (federated AP50 / centralized AP50 of the class, %, means over seeds, both calibrated). CWT and FedAsync: seeds 100–109 ($n = 10$). FedAvg: seeds 0–2. ρ: Spearman correlation of retention with the class's training boxes, counted in the images trained on (Table 2).

Scheme	Use case	Worst	Median	Below 85%	Nodes	ρ (boxes)
CWT	KFM12	75	100	1/12	92–101	$+0.69^{*}$
	WBC	94	104	0/7	94–114	$+0.32$
	DENTEX	98	99	0/32	99	-0.61^{*}
FedAsync	KFM12	52	98	3/12	81–101	$+0.57$
	WBC	111	131	0/7	126–137	-0.11
	DENTEX	97	100	0/32	99–100	$+0.34$
FedAvg	KFM12	59	98	3/12	87–94	$+0.69^{*}$
(seeds 0–2)	WBC	89	112	0/7	103–142	$+0.07$

* $p < 0.05$ (permutation test, 20,000 permutations; FedAsync: $p = 0.06$ on KFM12 and $p = 0.05$ on DENTEX). Nodes: range of the mean retention of each node's classes. On DENTEX retention spans only 98–100% (CWT) and 97–102% (FedAsync), so its correlations describe differences of a few points.

in the evaluation set, its AP is the least precise of the study. Retention on KFM12 still grows with class size ($\rho = +0.69$). On DENTEX its worst class is at 98%.

- *FedAsync.* It concentrates its KFM12 loss on the three rarest classes, while its median class is at 98%: eimeria at 52%, metastrongili (about 32 training boxes per seed) at 59% and dicrocoelium (about 22) at 69%. All three paired differences exclude zero. The two nodes that own these classes average 84% and 81%; the other two average 95% and 101%. On WBC FedAsync scores above centralized on every class (111–168%), an excess that weight averaging does not explain (Section 5.1). On DENTEX it keeps every class at 97% or more and every node at 99–100%.
- *FedAvg.* On seeds 0–2 it concentrates its losses on rare classes in the same way: worst class 59% on KFM12, $\rho = +0.69$.

A federation that meets a mean-level guarantee can thus still fail the owners of rare classes. DENTEX has no rare class (262–506 training boxes per tooth), and neither scheme loses a class there. Reporting the worst class and the per-node retention next to the mean should be part of the acceptance criteria.

5.2.2 Compute, communication and waiting time

All federated schemes train the same number of node-epochs, so their compute is the same. Every run was trained on one laptop GPU (NVIDIA RTX 3080 Laptop, 8 GB), which ran the nodes one after another. Time and energy below are measured on that GPU; communication is computed; waiting time is derived from the timing model. None of them is a measurement of a deployment across sites.

- *Time (measured).* A federated run took a median of 26–62 GPU minutes depending on the use case, plus 4–11 minutes of calibration. This is 1.1–1.4 times the centralized run on the same data, the difference being per-turn overhead. It is the total compute of the four nodes, not the time a deployment would take.
- *Energy (measured, GPU only).* Integrated from the GPU board's power draw, read by `nvidia-smi` every 10 s during the ten-seed study (median 71 W while training), a

Table 7 Comparison with standard methods (seeds 0–2 for every method on KFM12 and WBC, and seeds 100–102 on DENTEX). *Fed:Cent*: mean, with the lower end of its 95% CI in parentheses; worst: retention of the class with the lowest federated / centralized AP50. Post-processing: closing calibration after parallel and sequential training, progressive calibration with refresh after concurrent training, applied to the centralized reference as well. Three seeds, no test. On DENTEX the COFY FedAsync and CWT rows are the runs of Table 5 for those seeds.

		KFM12		WBC		DENTEX	
Method	Execution	*Fed:Cent*	Worst	*Fed:Cent*	Worst	*Fed:Cent*	Worst
COFY FedAvg	parallel	94.0 (79.6)	59	114.8 (94.2)	89	99.3 (97.8)	92
FedProx	parallel	93.8 (82.9)	45	123.6 (77.7)	107	99.1 (98.2)	94
FedBuff	concurrent	96.3 (81.1)	47	122.6 (90.6)	107	99.1 (94.7)	96
CA2FL	concurrent	87.1 (81.7)	41	104.2 (89.0)	89	95.8 (93.1)	90
COFY FedAsync	concurrent	96.2 (89.3)	68	117.5 (70.4)	108	99.9 (99.4)	95
COFY CWT	sequential	102.1 (85.7)	97	111.0 (98.8)	92	100.4 (99.1)	98

run used 0.03–0.08 kWh. One seed of all three arms on KFM12, WBC and DENTEX used about 0.52 kWh. The processor, memory, storage and network are not included.

- *DENTEX.* Every node holds every image, so each image is trained four times per epoch, once by each owner with its own labels. The centralized reference does the same (Table 3), so the two train the same number of image-epochs, as on the other use cases.
- *Communication (computed).* It is the same for all schemes: one download and one upload of the 99.4 MB model per node and turn, which totals 2.0–3.0 GB per node. The calibration adds 0.28 MB of statistics and rows per node. These volumes follow from the size of the model in 32-bit floating point and the number of transfers, without protocol overhead or compression; they were not measured on a network.

The schemes differ in waiting time (derived). With the machine speeds of our timing model, CWT needs 2.4–3.3 times the wall-clock time of FedAsync, and FedAvg about the same as FedAsync (1.00–1.03). These ratios count computation only (Section 4.1). CWT waits for the sum of every node's turns, FedAvg for the slowest node in each round, and FedAsync for its last arrival. Real transfer times, failures and differences in hardware larger than our ±30% would change them. CWT also exposes each node's individual update to the server. It therefore cannot use secure aggregation [65], which hides individual updates from the server and is available to FedAvg.

5.3 Comparison with standard methods and a local-only baseline

Table 7 compares the schemes with FedProx [16], FedBuff [36] and CA2FL [37] on seeds 0–2 of KFM12 and WBC and on seeds 100–102 of DENTEX. The standard methods run inside the same system, with the same masked loss, class-row ownership, training budget and post-processing; only the rule that combines updates changes. FedProx adds a proximal term ($\mu = 0.01$) to FedAvg. FedBuff and CA2FL replace the merge of FedAsync with buffered aggregation, and CA2FL also calibrates each step with a cache of every node's latest update.

Table 8 Local-only baseline: each node trains alone on its own data and classes, and each class is scored on its owner's model. Seeds 100–102 for every row, so the federated rows differ from the ten-seed values of Tables 5 and 6. Ratios to the calibrated centralized reference of the same seed (%). *Fed:Cent*: mean, with the lower end of its 95% CI in parentheses. Worst class, classes below 85% and node range as in Table 6. Three seeds, no test.

Use case	Training	*Fed:Cent*	Worst class	Below 85%	Nodes
KFM12	local only	89.5 (79.9)	34	4/12	75–94
	CWT	98.8 (97.6)	73	1/12	91–104
	FedAsync	93.1 (86.7)	49	3/12	77–106
WBC	local only	95.9 (76.0)	81	1/7	81–107
	CWT	103.1 (89.3)	81	1/7	99–127
	FedAsync	136.4 (86.0)	108	0/7	140–153
DENTEX	local only	99.9 (98.5)	96	0/32	99–101
	CWT	100.4 (99.1)	98	0/32	100–101
	FedAsync	99.9 (99.4)	95	0/32	99–100

- *KFM12.* CWT is the only method above centralized, and the only one that keeps every class above 85% (worst class 97%). Every method that merges updates trained in parallel or concurrently loses the rarest class, eimeria, keeping 41–68% of its centralized AP50. Among the standard methods, FedBuff has the highest mean (96.3%) and CA2FL the lowest (87.1%).
- *WBC.* Every method scores above centralized on average (104–124%), and with three seeds the confidence bounds overlap widely.
- *DENTEX.* Every method stays close to centralized (95.8–100.4%), with every tooth at 90% or more. CA2FL is the lowest; CWT is the highest, with its worst tooth at 98%.

In these runs, buffering and update caches did not remove the rare-class loss of averaging-based schemes; only server-scheduled turns did. With three seeds, one setting per method (for example $\mu = 0.01$ for FedProx) and no test, this comparison ranks methods only roughly. It does not show that COFY's schemes are generally better than FedProx, FedBuff or CA2FL, which here differ from them only in the merge rule; the claims of Section 5.1 rest on Table 5 alone.

Table 8 compares training alone with the two federated schemes on the same seeds.

- *KFM12.* Training alone keeps 89.5% of centralized, against 98.8% for CWT. The difference falls on the rare classes. Eimeria keeps 34% of its centralized AP50 when its owner trains alone, 73% under CWT and 49% under FedAsync. Under CWT every node keeps more than it does alone (91–104% against 75–94%). Under FedAsync one node does not: the owner of dicrocoelium and metastrongili keeps 89% alone and 77% in the federation.
- *WBC.* Training alone keeps 95.9%, and both schemes score higher. The node that owns neutrophils gains most from federating: 81% alone, 106% under CWT. With three seeds the bounds are wide.
- *DENTEX.* Training alone matches centralized (99.9%), as both schemes do. Every node holds every radiograph, so each owner already trains on all the images of its teeth. Federation adds no accuracy here, only one detector in place of four.

Table 9 Robustness on WBC: *Fed:Cent* (%, mean over seeds 0–2, attacker rotating with the seed), without / with the defence, last global version without calibration. –: not run. Row hijacking is shown with ownership not enforced / enforced, without the other parts of the defence.

Attack	CWT	FedAsync	FedAvg
none (cost of the defence)	106 / 100	109 / 110	112 / 103
noise	80 / 82	97 / 98	–
box corruption	83 / 82	98 / 98	–
BatchNorm poisoning	53 / 100	75 / 70[a]	–
scaling ×10	22 / 84	34 / 81	34 / 121
weight inflation ×100	–	–	61 / 116
row hijacking	90 / 101	90 / 112	–
phantom-object backdoor[b]	96 / 109	114 / 106	112 / 110
sign flip	0 / 0	0 / 5	0 / 0
sign flip, per-class budget[c]	– / 82	– / 91	– / 96

[a] One seed at 0 both without and with the defence. [b] Attack success rate 100% in every run, with and without the defence. [c] The defended sign-flip runs repeated with their final model kept and scored with a per-class detection budget (Section 4.4); the attacker's own classes score 0.

The local-only reference favours not federating. A node that trains alone detects only its own classes, and the scoring above assumes that the four models are exchanged after training and run side by side, at four times the inference cost. Federation adds the most accuracy where classes are rare and spread over nodes, and none where every node sees every image. FedAsync can leave a node worse off than training alone. Each node can check this on its own validation images, by comparing the federated model with one it trained alone on its own classes, without data from the other nodes.

5.4 Robustness to a malicious node

Table 9 summarizes the robustness study.

- *Attacks that enlarge an update are neutralized.* Scaling an update tenfold leaves the undefended federation at 22–34% of centralized, and at 81–121% with the defence. Weight inflation in FedAvg goes from 61% to 116%, and BatchNorm poisoning in CWT from 53% to 100%. On clean federations the defence costs nothing measurable: 106% / 100%, 109% / 110% and 112% / 103%, within the seed-to-seed spread.
- *Ownership blocks row hijacking.* With class rows taken from the registry, an attacker that claims every class has no effect (101% and 112%); without it, accuracy falls to 90% in both schemes.
- *Attacks on the attacker's own data mostly harm its own classes.* Noise brings the attacker's classes to mAP50 0.02–0.09, against 0.39–0.49 in clean runs. With the defence, the other nodes' classes stay within 5% of a defended clean run, or above it; without the defence, in CWT, they lose 10–14%. Damage confined to a node's own classes cannot be detected by any party that lacks data for those classes.
- *The backdoor succeeds in every scheme, with and without the defence.* Accuracy stays intact (96–114% of centralized), and the attacker's class is detected on the trigger in every triggered evaluation image (2,016–2,492 per seed), in all eighteen runs. The clean runs, which never saw the trigger, detect it on at most 1.2% of the images. The defence scaled none of the attacker's uploads in CWT and FedAvg and

23% of them in FedAsync, without effect. The backdoor is an update of the size of an honest one, to class rows that only the attacker trains, and no party without data for that class can tell it from honest training. We did not test a backdoor aimed at another node's class. The registry discards the attacker's changes to that class's rows, but not to the shared layers.

- *Sign flip defeats every scheme, with and without the defence.* mAP50 is 0 on every seed and scheme without the defence, and at most 0.056 with it. The reversed update has the size of an honest one, so the norm bounds do not touch it. In the shared layers, the honest nodes' updates pull the model back. The attacker's class rows have no other contributor, so the reversal accumulates over rounds, and the bounds, which act on one update at a time, cannot stop a direction that persists. Reversing the training of a class raises its scores on background, where the loss lowers them: the class fires everywhere. With the shared budget of 300 detections per image, the attacker's class then fills every place (Figure 5h), and every other class scores zero. The repeated runs confirm this: they collapse again under standard evaluation (0–25% of centralized), but scored with a per-class budget the other nodes' classes reach mAP50 0.54–0.69, as in defended clean runs (0.54–0.69), and the federation 82%, 91% and 96% of centralized in the three schemes. Only the attacker's own classes are lost, which no party without data for them could prevent. On clean runs the per-class budget changes nothing (identical scores on all nine). Single ownership is what makes a per-class bound possible, and it is also what this attack exploits. A per-class output budget contains the harm within the attacker's classes, without repairing the model.
- *BatchNorm poisoning in FedAsync* is not stopped: 75% without the defence and 70% with it, because one seed collapses to zero in both cases, and a per-class budget does not help, since the poisoned statistics corrupt the features of every class. The defence did scale each of the attacker's ten uploads down to 4–13% of its size. In this seed, however, the attacker was the slowest node and sent its last three updates in a row at the end of training, all pushing the statistics the same way, with no honest update after them. As with the sign flip, bounds on single updates do not stop a persistent direction.

Table 10 summarizes the candidate defences, each tested in a few post hoc runs on WBC. The calibration undoes BatchNorm poisoning in training because the poisoning lies in the running statistics, which training does not use and the calibration recomputes; the final model, which averages the global versions right after each node's last update (Section 3.2), keeps the attacker's last poisoned statistics until then. Median pooling holds where the mean collapses, because one node among four cannot move a coordinate-wise median outside the range of the honest ones. Screening fails where a node's own detections are about as stable under blending as the trigger (thresholds 0.001–0.09, trigger entropy 0.02–0.20): a threshold taken from a node's own detections does not carry over to the other nodes' classes, so its false-alarm rate cannot yet be controlled. The joint bound fails because one bounded update of the statistics is enough when no honest update follows: in the seed whose attacker sent its last updates in a row, the last global version collapsed again (mAP50 0.009). Peer validation found the attacker's classes in every sign-flip run and rolled back only them (node C's second

Table 10 Candidate defences, tested after the results on WBC, seeds 0–2, against the attacks that the defence did not stop. Accuracy as *Fed:Cent* (%): final models after the BatchNorm step of the calibration (rows 1–2), last global version (rows 4–6, CWT / FedAsync / FedAvg); row 3: the backdoor's attack success rate.

Attack	Defence	Detection	Accuracy restored	Cost
BatchNorm poisoning in training	BatchNorm step of the calibration (the study's rule, count-weighted)	not needed: the statistics are recomputed	12 of 12 poisoned final models, to 74–120 (uncalibrated: mAP50 0.000–0.57)	none
BatchNorm poisoning in training and in the calibration	median pooling of the BatchNorm statistics	not needed	21 of 21 models, 75–120 (mean pooling: mAP50 0.000 in all 21)	at most 0.006 mAP50 when every node is honest
phantom-object backdoor	screening inputs at the nodes	15 of 18 runs; AUROC 0.88–1.00, in the failures 0.57–0.78	success rate 100% → 0–3% in 15 of 18 runs	0–41% of clean images flagged (median 7%)
BatchNorm poisoning, FedAsync	joint bound of consecutive merges	–	none: 69, against 70 without	none on clean runs (115 against 109)
sign flip	peer validation with per-class rollback	9 of 9 runs, at the 3rd–17th check; no false alarm in 9 clean runs	31 / 17 / 91, against 0 / 1 / 0; with a per-class budget 75–91	none on clean runs (98 / 110 / 115, against 100 / 109 / 103); one number per class and check disclosed
sign flip	peer validation, rollback and exclusion of the flagged node	9 of 9 runs, at the 3rd–30th check; acts only after a rollback	77 / 97 / 97, against 31 / 17 / 91 with rollback alone; other nodes' classes mAP50 0.49–0.75 (clean runs 0.55–0.68)	none on clean runs, which had no rollback; a node excluded by mistake would lose all its classes

class at the 36th check in one FedAsync seed). The node named as attacker, which reported every class it does not own as covering every location, cast no vote: such a report fails the ratio to the other classes, and a report aimed at a single class is one vote of three, short of a majority. Under FedAvg the other nodes' classes reach mAP50 0.59–0.64, as in clean runs, because its trimmed mean of the shared parameters discards the attacker's reversed updates. Under CWT and FedAsync these updates are still merged, bounded only in size, and by the end of training the frozen rows fire again on 3–29% of the locations. Excluding the flagged node from then on sets these updates aside. It lifted CWT from 31% to 77% of centralized, FedAsync from 17% to 97% and FedAvg from 91% to 97%. The other nodes' classes reach mAP50 0.55–0.75 under FedAsync and 0.58–0.72 under FedAvg, as in clean runs (0.61–0.68). Under CWT they reach 0.49–0.57, and two seeds stay 0.06–0.07 below their clean runs. The attacker's classes stay at zero, as with every defence we tested: no other node has data to train them. The frozen rows still fire at the end of training, on up to 15% of the locations in CWT and FedAsync, since the shared layers keep changing under them.

Table 11 Evidence behind each finding. Main study: run once on ten seeds not used to design the schemes, with code, configurations and analysis frozen before the first run. Descriptive: reported without a test. Post hoc: designed after seeing the results it follows.

Finding	Evidence	Basis
CWT keeps at least 95% on KFM12, WBC and DENTEX (Table 5)	main study	ten fresh seeds; frozen code and analysis
FedAsync keeps at least 90% on the same use cases	main study	the same runs and analysis
Lower bounds under four other intervals (Appendix A.4)	post hoc	two below 95% for CWT on DENTEX
Per-class and per-node retention (Table 6)	descriptive	seeds 100–109; FedAvg: seeds 0–2
Standard methods (Table 7)	descriptive	three seeds, one setting per method
Local-only baseline, averaged centralized reference	descriptive, post hoc	added after the main results
GPU time and energy	measured	one GPU running the simulation; GPU board only
Communication, waiting time	computed, modelled	model size and transfers; timing model
Robustness study (Table 9)	descriptive	WBC, three seeds, one non-colluding attacker
Per-class detection budget	post hoc	added after the sign-flip results
Candidate defences (Table 10)	post hoc	designed after the robustness results; same seeds; thresholds from pilots

They no longer crowd out the other classes, though: a per-class budget changes the final models by at most 0.004 mAP50.

6 Discussion

The results support a deployment choice between two operating points rather than a single best scheme. We discuss these operating points and their guarantees, how the system and its evaluation map onto requirements for trustworthy AI, which privacy guarantees each scheme can offer, and the limitations of the study.

Not every finding carries the same weight. Table 11 states the evidence behind each one: only the ten-seed study supports a claim, and every other result describes the runs reported here.

6.1 Two operating points, stated with their guarantees

The results point to two operating points rather than one best scheme:

- *CWT, sequential turns.* It keeps at least 95% of centralized mAP50 on KFM12, WBC and DENTEX. On KFM12, WBC and DENTEX every class but one stays within 85% of centralized. The price is about three times the wall-clock time, and individual updates visible to the server.
- *FedAsync, concurrent operation.* It needs no coordination and keeps the same communication cost. It keeps at least 90% on KFM12, WBC and DENTEX (Section 5.1). Its losses fall on the rarest classes: on KFM12, three classes lose 31–48% while the median class loses 2%. On the same use case, one node kept less accuracy under FedAsync than training alone (Section 5.3).

Stating each mode's bound, its worst class and its costs side by side lets the people who deploy and oversee the system choose with full knowledge. We see this as the practical meaning of transparency for a federated system. A FedAsync deployment should therefore start with a pilot on the target data, comparing its per-class and per-node retention with a CWT run, and each node should check that it gains from taking part, against a model trained alone on its own classes. Neither mode withstands a backdoor in the attacker's own class, or a node that reverses its updates under standard evaluation (Section 5.4). A deployment can contain the second attack by giving each class its own detection budget at inference. This does not repair the model, but a class that fires everywhere can no longer take the other classes' detections; it costs no accuracy on clean models, only more detections to post-process. A deployment should also monitor how often each class is detected on every node's own images, which any node can do without labels for the other classes. Four candidate defences, designed after the results and tested post hoc on WBC with three seeds, point further; none is established beyond these runs. The study's calibration already undoes BatchNorm poisoning during training. Pooling its statistics by their median cost at most 0.006 mAP50 and kept every model intact when the attacker poisoned the calibration too. Screening inputs at the nodes caught the backdoor in most runs, but at a false-alarm rate that a node cannot yet control from its own data. Bounding consecutive merges did not stop BatchNorm poisoning. Peer validation with per-class rollback found the class of a node reversing its updates in each of its nine runs, with no false alarm and no cost. It restored FedAvg; under CWT and FedAsync the frozen class fired again, through the attacker's updates to the shared layers. Excluding the flagged node from then on set those updates aside: CWT rose from 31% to 77% of centralized and FedAsync from 17% to 97%, and only the attacker's own classes were lost. Exclusion raises the stakes of a false alarm, since a node excluded by mistake loses all its classes; no honest class was rolled back in our 27 runs.

6.2 Alignment with requirements for trustworthy AI

Table 12 maps the measures of this work onto the requirements of the EU AI Act for high-risk systems [53], which also reflect the European guidelines [52] and the NIST AI RMF [54]. Medical image-detection software used for diagnosis is high-risk as a medical device. COFY is research software, and we claim support for these requirements, not compliance.

6.3 Privacy

Images and labels never leave a node. Weights do, and in the calibration so do per-channel BatchNorm means and variances. Gradient inversion [66, 67] is a known risk for individual updates. It is less studied for detectors trained over several local epochs, but cannot be excluded. Secure aggregation [65] protects only schemes that merge several updates at once, which here means FedAvg and buffered asynchronous rules. Client-level differential privacy [68] applies to every scheme, at an accuracy cost we have not measured. The choice of scheme is therefore also a choice of privacy guarantee, and it should be made explicitly.

Table 12 How the system and its evaluation support requirements for high-risk AI (EU AI Act).

Requirement	What this work provides
Risk management (Art. 9)	threat model and attack study, including the attacks the defence does not stop; measured failure modes (rare-class loss, per-node retention)
Data governance (Art. 10)	data never leave sites; documented preparation, splits and data fingerprints; class-ownership registry
Record-keeping (Art. 12)	per-update logs (node, staleness, version); defence decisions; persisted server state; timing schedules
Transparency (Art. 13)	accuracy cost with confidence bounds; per-class worst case; deviations and negative results disclosed
Human oversight (Art. 14)	operator dashboards; control over lost nodes and turns; developer-only evaluation
Accuracy, robustness, cybersecurity (Art. 15)	non-inferiority bounds on fresh seeds, with frozen code and analysis; robust aggregation; authentication, safe decoding, payload validation

6.4 Limitations

- *Simulated concurrency and deployment.* Concurrency was simulated with a timing model on one GPU, and the waiting times are derived from that model; GPU time and energy were measured on that GPU alone, and communication was computed. The network implementation reproduces the schemes, but it was exercised with four client processes on one machine, and a multi-site deployment remains to be measured.
- *Statistical scope.* The confidence intervals are over ten training seeds, on fixed datasets, splits and node partitions. They bound the cost of federation for these use cases, not for new sites, populations or acquisition protocols, and they are not a clinical non-inferiority claim. Under four alternative intervals the lower bounds stay above 90%, but CWT's 95% on DENTEX holds under three of the five (Appendix A.4); ten seeds remain a small sample.
- *Four nodes.* All federations had four nodes, and every class had exactly one owner.
- *The comparator.* The reference is a centralized model with the same partial labels and the same examples (Table 3). Partial labelling costs both models alike: on WBC, centralized reaches about 58% of a full-label model. The per-turn warm-up, the per-node momentum and the order of the data differ, and their effects are included in *Fed:Cent*.
- *Scope of the robustness study.* It used one use case (WBC), three seeds, four nodes, one non-colluding attacker and uncalibrated models, so its findings are limited demonstrations; the calibration was attacked only in the test of a candidate defence, and only in its BatchNorm step. Attacks confined to the attacker's own classes, such as the backdoor, cannot be detected during training by any party that lacks data for those classes. The candidate defences were tested after the results, on the same use case and seeds, the screening threshold was chosen after two pilot rules, and the thresholds of peer validation were set on the kept final models of the same study. Exclusion of the flagged node was added after the results of the rollback alone.

- *Analyses added afterwards.* Only the ten-seed study supports the claims; its code, configurations and analysis were frozen before its first run. The local-only baseline and the averaged centralized reference were added after its results and are descriptive.

7 Conclusions

Federated object detection across sites that annotate different classes can be trusted only when its cost is stated with confidence, broken down by class and node, and tested against misbehaving participants. On ten fresh seeds, with code and analysis frozen before the first run, COFY with CWT retained at least 95% of the accuracy of a centralized model trained on the same partial labels on parasite eggs, white blood cells and dental radiographs (lower bounds 96.2–98.7%), and FedAsync at least 90%. FedAsync needs no coordination and stays close to centralized on average, but its losses fall on the rarest classes, which a mean-level guarantee hides. In a smaller robustness study (one use case, three seeds, one non-colluding attacker), ownership-aware robust aggregation neutralized attacks that enlarge an update and, through the class registry, attacks on other nodes' classes. It did not stop a backdoor that a node plants in its own class, which succeeded on every triggered image, nor a node that reverses its own updates, which collapsed every scheme because no other node can check the rows it alone trains; a per-class detection budget at inference contained that damage within the node's classes, without repairing the model. Candidate defences, designed afterwards and tested post hoc on the same seeds, point to remedies that need confirmation: screening inputs at the nodes caught the backdoor in most runs, though not yet at a controlled false-alarm rate; the calibration undid BatchNorm poisoning during training, and pooling its statistics by their median withstood an attacker that poisons them; peer validation by the other nodes found the node reversing its updates in every run, and excluding it lifted CWT and FedAsync to 77% and 97% of centralized. In comparisons on three seeds, standard methods (FedProx, FedBuff, CA2FL) shared the rare-class loss of averaging-based schemes; only server-scheduled turns kept every class within 85% of centralized. Compared with nodes that train alone and exchange their models, federation added the most accuracy on parasite eggs, chiefly on the rarest class, and none on dental radiographs, where every node holds every image. Weight averaging explains the small excess of CWT over centralized on white blood cells, but not the larger excess of FedAsync. We recommend that federated detectors be accepted on a lower confidence bound computed on fresh seeds with frozen code, their worst class and per-node retention, and an explicit statement of the attacks they do not withstand.

Declarations

- **Funding.** No funding was received for conducting this study.
- **Competing interests.** The authors have no competing interests to declare that are relevant to the content of this article.
- **Ethics approval and consent to participate.** Human data: the study uses two public datasets of human images, WBC (peripheral blood smears) and DENTEX

(panoramic radiographs), which their creators released in de-identified form for research use, under the licences given under Data availability. No new human data were collected, no person was contacted, and no attempt was made to re-identify anyone. Animal data: the KFM12 images come from faecal samples collected for routine parasitological diagnosis (Section 3.4). No animal was handled, sampled or treated for this study, so no ethics approval was required.

- **Consent for publication.** Not applicable.
- **Data availability.** WBC: images of the PBC dataset [62] (Mendeley Data, version 1, CC BY 4.0) with the bounding boxes of [61] (CC BY 4.0); 13,344 images after conversion. DENTEX [63]: the 634 panoramic radiographs of the quadrant-enumeration subset of the challenge's training data, as released on Hugging Face (CC BY-NC-SA 4.0). KFM12: the 15,448 images with their twelve-class labels are released on Mendeley Data [Authors: DOI and licence]; a two-class subset is already public [35]. We redistribute no WBC or DENTEX images. The code release contains the conversion and split scripts, the SHA-256 fingerprints of every split and node, which `tools/rebuild_study_data.py` checks after rebuilding them from the original data, and the list of the images each node trained on in every run of Table 5.
- **Materials availability.** Not applicable.
- **Code availability.** The code (system, network implementation, study tools), every configuration, the evaluation protocol, the frozen code and analysis script with their SHA-256 records, and the run-level records and seed-level values of the runs reported here are available at https://github.com/SalvatoreCapuozzo/COFY under the GNU AGPL-3.0 licence, together with a one-command demonstration that runs a four-node federation over the network on sample images of the three use cases. The release used for this article is archived on Zenodo [Authors: DOI of the archived release].
- **Author contribution.** Salvatore Capuozzo: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft, Writing – review & editing. Stefano Marrone: Conceptualization, Supervision, Writing – review & editing. Carlo Sansone: Supervision, Writing – review & editing. All authors read and approved the final manuscript.
- **Use of generative AI.** The authors designed the system architecture, the study and its experiments. Claude Opus 5.5 (Anthropic) was used to draft the manuscript, to develop part of the code, and to automate the experimental runs (Section 4). The code it developed implements components conceived and designed by the authors, who also validated it. The authors reviewed and verified all content and take full responsibility for it.

Appendix A Reproducibility

Paths below are relative to the `cofy_system` directory of the code release (https://github.com/SalvatoreCapuozzo/COFY, archived as stated under Code availability).

A.1 Data

DENTEX is converted to YOLO format by `tools/convert_dentex.py`, which keeps every box and re-encodes the images as JPEG (quality 95) at full resolution; it reads only the quadrant-enumeration folder of the challenge's training data. WBC is converted by `tools/coco_to_yolo.py`. Every dataset is split by `tools/split_dataset_into_nodes.py --split-mode global --n-nodes 4 --seed 0 --val-frac 0.2`. On a new machine, `tools/rebuild_study_data.py` re-runs the split with these arguments. It then checks all 138 fingerprints of the study datasets against `analysis/study_manifest.json` (SHA-256 `708fde22...b5a650`); the DENTEX fingerprints are in its own manifest (below). The training subsets are drawn by `cofy/sampling.py` from the seed, and every run writes the list it trained on. Tables 1, 2 and 3 are computed from these lists by `tools/dataset_tables.py`.

A.2 Frozen study

The schemes, configurations, training, calibration and evaluation code and the analysis script were frozen on 28 October 2026 in `_frozen/confirm/`. Their SHA-256 values are listed in `analysis/confirm_manifest.json` (94 files; the file's own SHA-256 is `0d1a8b8f...4857e0`). The evaluation protocol of the study, as reported here, is in `analysis/EVALUATION_PROTOCOL.md`. The DENTEX configurations and label fingerprints are listed in `analysis/confirm_manifest_amend1.json` (`5aceca94...a75033`). `tools/confirm_manifest.py --check` recomputes them. It reads code from the frozen copy and lists separately the files recorded later, the DENTEX configurations. The study runs from that copy through `runs/variants/_queue.sh`, which executes `runs/variants/_plan.txt` line by line. The plan holds one line per arm, use case and seed. `tools/export_evidence.py` copies the records of the runs reported here into `evidence/study/`: per seed, the calibrated scores of every arm, each run's summary, FedAsync's timeline and the images each node trained on.

A.3 Tables and follow-up analyses

- Table 5: intervals from `tools/confirm_analysis.py --write`, run from the frozen copy. Levels of Section 5.1 and Tables A1 and A2: `tools/study_sensitivity.py`.
- Table 6: `tools/fairness_analysis.py --confirm`, and without `--confirm` for the FedAvg rows (seeds 0–2); class sizes are counted in the images each run trained on.
- Cost (Section 5.2): `tools/cost_report.py`, from the run logs and the GPU power log.
- Robustness (Table 9): `tools/robust_eval.py --report`.
- Local-only baseline, averaged centralized reference and per-class-budget evaluation: `tools/followup_runs.py`.
- Candidate defences: `tools/defence_eval.py`, `cofy/robust.py` (joint bound), `cofy/peercheck.py` (peer validation), rules check `tools/check_rules.py`; Figure 5: `tools/make_attack_figure.py`.

Every run keeps its configuration, log and summary. FedAsync runs also keep their timeline (`virtual_schedule.json`), and defended runs the defence's decisions (`robust_log.jsonl`).

A.4 Seed-level results and sensitivity

Table A1 lists the paired values behind Table 5: the calibrated mAP50 of the centralized reference and of both schemes on each of the ten seeds (`analysis/study_seed_level.csv`; per-class values and the training subsets in `evidence/study/`). Table A2 recomputes the lower bounds of Table 5 with four other intervals, as a check (`tools/study_sensitivity.py`). The log-ratio interval drops the assumption that the ratio is symmetric; the bootstrap and the Wilcoxon interval drop normality, the latter at an exact confidence of 95.1% for ten seeds; the leave-one-out bound shows how much a single seed moves the result. With ten seeds, bootstrap intervals tend to be slightly too narrow.

Table A1 Calibrated mAP50 on each of the ten seeds (100–109). Centralized: the reference both schemes are paired with on that seed.

Use case	Model	100	101	102	103	104	105	106	107	108	109
KFM12	centralized	.736	.700	.740	.729	.723	.744	.740	.730	.752	.744
	CWT	.731	.689	.730	.738	.743	.724	.734	.730	.698	.719
	FedAsync	.675	.673	.677	.705	.687	.686	.682	.699	.689	.693
WBC	centralized	.463	.506	.490	.462	.568	.475	.504	.494	.549	.511
	CWT	.502	.493	.507	.549	.553	.587	.508	.529	.527	.514
	FedAsync	.738	.618	.625	.630	.692	.649	.680	.647	.650	.610
DENTEX	centralized	.959	.953	.960	.962	.961	.950	.958	.960	.963	.960
	CWT	.961	.962	.960	.961	.959	.944	.847	.958	.964	.956
	FedAsync	.957	.955	.958	.957	.956	.951	.960	.958	.950	.957

Table A2 Lower end of 95% intervals for the mean *Fed:Cent* (%), with the t-interval of Eq. (2) first. Log-t: t-interval of the log ratio, back-transformed. BCa: bias-corrected and accelerated bootstrap (20,000 resamples). HL: exact Wilcoxon signed-rank interval of the Hodges–Lehmann estimate. LOO: smallest t lower bound with one seed left out. Level: the share each scheme keeps on every use case (Section 5.1).

Scheme	Use case	Mean	t, Eq. (2)	Log-t	BCa	HL	LOO	Level
CWT	KFM12	98.6	96.7	96.6	96.8	96.9	96.2	95
	WBC	105.4	98.7	98.7	100.9	99.0	97.3	95
	DENTEX	98.8	96.2	96.0	94.4	94.7	95.6	95
FedAsync	KFM12	93.6	92.1	92.1	92.5	91.9	91.8	90
	WBC	130.8	122.0	122.2	125.2	122.1	120.2	90
	DENTEX	99.7	99.4	99.4	99.3	99.4	99.3	90

References

[1] McMahan, H.B., Moore, E., Ramage, D., Hampson, S., Arcas, B.: Communication-efficient learning of deep networks from decentralized data. In: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS). Proceedings of Machine Learning Research, vol. 54, pp. 1273–1282 (2017). https://doi.org/10.48550/arXiv.1602.05629

[2] Kairouz, P., McMahan, H.B., *et al.*: Advances and open problems in federated learning. Foundations and Trends in Machine Learning **14**(1–2), 1–210 (2021) https://doi.org/10.1561/2200000083

[3] Rieke, N., Hancox, J., Li, W., Milletarì, F., Roth, H.R., Albarqouni, S., Bakas, S., Galtier, M.N., Landman, B.A., Maier-Hein, K., Ourselin, S., Sheller, M., Summers, R.M., Trask, A., Xu, D., Baust, M., Cardoso, M.J.: The future of digital health with federated learning. npj Digital Medicine **3**, 119 (2020) https://doi.org/10.1038/s41746-020-00323-1

[4] Sheller, M.J., Edwards, B., Reina, G.A., *et al.*: Federated learning in medicine: Facilitating multi-institutional collaborations without sharing patient data. Scientific Reports **10**, 12598 (2020) https://doi.org/10.1038/s41598-020-69250-1

[5] Kaissis, G.A., Makowski, M.R., Rückert, D., Braren, R.F.: Secure, privacy-preserving and federated machine learning in medical imaging. Nature Machine Intelligence **2**(6), 305–311 (2020) https://doi.org/10.1038/s42256-020-0186-1

[6] Dayan, I., Roth, H.R., Zhong, A., *et al.*: Federated learning for predicting clinical outcomes in patients with COVID-19. Nature Medicine **27**(10), 1735–1743 (2021) https://doi.org/10.1038/s41591-021-01506-3

[7] Bouthillier, X., Delaunay, P., Bronzi, M., Trofimov, A., Nichyporuk, B., Szeto, J., Mohammadi Sepahvand, N., Raff, E., Madan, K., Voleti, V., Ebrahimi Kahou, S., Michalski, V., Serdyuk, D., Arbel, T., Pal, C., Varoquaux, G., Vincent, P.: Accounting for variance in machine learning benchmarks. In: Proceedings of Machine Learning and Systems (MLSys), vol. 3 (2021). https://doi.org/10.48550/arXiv.2103.03098

[8] Li, X.-C., Zhan, D.-C.: FedRS: Federated learning with restricted softmax for label distribution non-IID data. In: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), pp. 995–1005 (2021). https://doi.org/10.1145/3447548.3467254

[9] Xu, X., Deng, H.H., Gateno, J., Yan, P.: Federated multi-organ segmentation with inconsistent labels. IEEE Transactions on Medical Imaging **42**(10), 2948–2960 (2023) https://doi.org/10.1109/TMI.2023.3270140

[10] Chang, K., Balachandar, N., Lam, C., Yi, D., Brown, J., Beers, A., Rosen, B.,

Rubin, D.L., Kalpathy-Cramer, J.: Distributed deep learning networks among institutions for medical imaging. Journal of the American Medical Informatics Association **25**(8), 945–954 (2018) https://doi.org/10.1093/jamia/ocy017

[11] Xie, C., Koyejo, S., Gupta, I.: Asynchronous federated optimization. arXiv preprint arXiv:1903.03934 (2019) https://doi.org/10.48550/arXiv.1903.03934

[12] Izmailov, P., Podoprikhin, D., Garipov, T., Vetrov, D., Wilson, A.G.: Averaging weights leads to wider optima and better generalization. In: Proceedings of the 34th Conference on Uncertainty in Artificial Intelligence (UAI) (2018). https://doi.org/10.48550/arXiv.1803.05407

[13] Luo, M., Chen, F., Hu, D., Zhang, Y., Liang, J., Feng, J.: No fear of heterogeneity: Classifier calibration for federated learning with non-IID data. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 34 (2021). https://doi.org/10.48550/arXiv.2106.05001

[14] Piaggio, G., Elbourne, D.R., Pocock, S.J., Evans, S.J.W., Altman, D.G.: Reporting of noninferiority and equivalence randomized trials: Extension of the CONSORT 2010 statement. JAMA **308**(24), 2594–2604 (2012) https://doi.org/10.1001/jama.2012.87802

[15] Berger, R.L., Hsu, J.C.: Bioequivalence trials, intersection-union tests and equivalence confidence sets. Statistical Science **11**(4), 283–319 (1996) https://doi.org/10.1214/ss/1032280304

[16] Li, T., Sahu, A.K., Zaheer, M., Sanjabi, M., Talwalkar, A., Smith, V.: Federated optimization in heterogeneous networks. In: Proceedings of Machine Learning and Systems (MLSys), vol. 2, pp. 429–450 (2020). https://doi.org/10.48550/arXiv.1812.06127

[17] Karimireddy, S.P., Kale, S., Mohri, M., Reddi, S.J., Stich, S.U., Suresh, A.T.: SCAFFOLD: Stochastic controlled averaging for federated learning. In: Proceedings of the 37th International Conference on Machine Learning (ICML). Proceedings of Machine Learning Research, vol. 119, pp. 5132–5143 (2020). https://doi.org/10.48550/arXiv.1910.06378

[18] Wang, J., Liu, Q., Liang, H., Joshi, G., Poor, H.V.: Tackling the objective inconsistency problem in heterogeneous federated optimization. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 33 (2020). https://doi.org/10.48550/arXiv.2007.07481

[19] Li, X., Jiang, M., Zhang, X., Kamp, M., Dou, Q.: FedBN: Federated learning on non-IID features via local batch normalization. In: International Conference on Learning Representations (ICLR) (2021). https://doi.org/10.48550/arXiv.2102.07623

[20] Yu, F.X., Rawat, A.S., Menon, A.K., Kumar, S.: Federated learning with only positive labels. In: Proceedings of the 37th International Conference on Machine Learning (ICML). Proceedings of Machine Learning Research, vol. 119 (2020). https://doi.org/10.48550/arXiv.2004.10342

[21] Fan, Z., Zhang, R., Yao, J., Han, B., Zhang, Y., Wang, Y.: Federated learning with bilateral curation for partially class-disjoint data. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 36, pp. 32006–32019 (2023). https://doi.org/10.52202/075280-1389

[22] Deng, Z., Luo, L., Chen, H.: Scale federated learning for label set mismatch in medical image classification. In: Medical Image Computing and Computer Assisted Intervention – MICCAI 2023. Lecture Notes in Computer Science, pp. 118–127. Springer, ??? (2023). https://doi.org/10.1007/978-3-031-43898-1_12

[23] Wang, P., Shen, C., Wang, W., Oda, M., Fuh, C.-S., Mori, K., Roth, H.R.: ConDistFL: Conditional distillation for federated learning from partially annotated data. In: Medical Image Computing and Computer Assisted Intervention – MICCAI 2023 Workshops. Lecture Notes in Computer Science, pp. 311–321. Springer, ??? (2023). https://doi.org/10.1007/978-3-031-47401-9_30

[24] Wagner, F., Xu, W., Saha, P., Liang, Z., Whitehouse, D., Menon, D., Newcombe, V., Voets, N., Noble, J.A., Kamnitsas, K.: Feasibility of federated learning from client databases with different brain diseases and MRI modalities. In: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), pp. 357–367 (2025). https://doi.org/10.1109/WACV61041.2025.00045

[25] Liu, Y., Huang, A., Luo, Y., Huang, H., Liu, Y., Chen, Y., Feng, L., Chen, T., Yu, H., Yang, Q.: FedVision: An online visual object detection platform powered by federated learning. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 34, pp. 13172–13179 (2020). https://doi.org/10.1609/aaai.v34i08.7021

[26] Luo, J., Wu, X., Luo, Y., Huang, A., Huang, Y., Liu, Y., Yang, Q.: Real-world image datasets for federated learning. arXiv preprint arXiv:1910.11089 (2019) https://doi.org/10.48550/arXiv.1910.11089

[27] Li, Y., Kundu, S.S., Boels, M., Mahmoodi, T., Ourselin, S., Vercauteren, T., Dasgupta, P., Shapey, J., Granados, A.: UltraFlwr – an efficient federated surgical object detection framework. arXiv preprint arXiv:2503.15161 (2025) https://doi.org/10.48550/arXiv.2503.15161

[28] Rashidi, G., Bounias, D., Bujotzek, M., Martínez Mora, A., Neher, P., Maier-Hein, K.H.: The potential of federated learning for self-configuring medical object detection in heterogeneous data distributions. Scientific Reports **14**, 23844 (2024) https://doi.org/10.1038/s41598-024-74577-0

[29] Yamada, A., Hanaoka, S., Takenaga, T., Miki, S., Yoshikawa, T., Nomura, Y.: Investigation of distributed learning for automated lesion detection in head MR images. Radiological Physics and Technology **17**(3), 725–738 (2024) https://doi.org/10.1007/s12194-024-00827-5

[30] Cherukuri, K.S., Sha, K., Huang, Z.: Enabling Federated Object Detection for Connected Autonomous Vehicles: A Deployment-Oriented Evaluation. arXiv:2509.01868 (2025). https://doi.org/10.48550/arXiv.2509.01868

[31] Zhao, X., Schulter, S., Sharma, G., Tsai, Y.-H., Chandraker, M., Wu, Y.: Object detection with a unified label space from multiple datasets. In: Computer Vision – ECCV 2020. Lecture Notes in Computer Science, pp. 178–193. Springer, ??? (2020). https://doi.org/10.1007/978-3-030-58568-6_11

[32] Cringoli, G., Amadesi, A., Maurelli, M.P., Celano, B., Piantadosi, G., Bosco, A., Ciuca, L., Cesarelli, M., Bifulco, P., Montresor, A., Rinaldi, L.: The Kubic FLOTAC microscope (KFM): a new compact digital microscope for helminth egg counts. Parasitology **148**(4), 427–434 (2021) https://doi.org/10.1017/S003118202000219X

[33] Capuozzo, S., Marrone, S., Gravina, M., Cringoli, G., Rinaldi, L., Maurelli, M.P., Bosco, A., Orrù, G., Marcialis, G.L., Ghiani, L., Bini, S., Saggese, A., Vento, M., Sansone, C.: Automating parasite egg detection: Insights from the first AI-KFM challenge. Frontiers in Artificial Intelligence **7**, 1325219 (2024) https://doi.org/10.3389/frai.2024.1325219

[34] Capuozzo, S., Maurelli, M.P., Marrone, S., Celano, B., Martone, G., Vitiello, P., Hammami, I., Bosco, A., Ciuca, L., Cringoli, G., Sansone, C., Rinaldi, L.: A dedicated deep learning workflow for automatic *Fasciola hepatica* and *Calicophoron daubneyi* egg detection using the Kubic FLOTAC microscope. International Journal for Parasitology **56**(2), 104719 (2026) https://doi.org/10.1016/j.ijpara.2025.08.007

[35] Capuozzo, S.: Trematodes AI-KFM Dataset. Mendeley Data, V1 (2025). https://doi.org/10.17632/6bjcxw8dx2.1

[36] Nguyen, J., Malik, K., Zhan, H., Yousefpour, A., Rabbat, M., Malek, M., Huba, D.: Federated learning with buffered asynchronous aggregation. In: Proceedings of the 25th International Conference on Artificial Intelligence and Statistics (AISTATS). Proceedings of Machine Learning Research, vol. 151, pp. 3581–3607 (2022). https://doi.org/10.48550/arXiv.2106.06639

[37] Wang, Y., Cao, Y., Wu, J., Chen, R., Chen, J.: Tackling the data heterogeneity in asynchronous federated learning with cached update calibration. In: International Conference on Learning Representations (ICLR) (2024). No DOI or arXiv record. https://iclr.cc/virtual/2024/poster/19456

[38] Blanchard, P., El Mhamdi, E.M., Guerraoui, R., Stainer, J.: Machine learning with adversaries: Byzantine tolerant gradient descent. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 30, pp. 119–129 (2017). No DOI or arXiv record. https://proceedings.neurips.cc/paper/2017/hash/f4b9ec30ad9f68f89b29639786cb62ef-Abstract.html

[39] Yin, D., Chen, Y., Ramchandran, K., Bartlett, P.: Byzantine-robust distributed learning: Towards optimal statistical rates. In: Proceedings of the 35th International Conference on Machine Learning (ICML). Proceedings of Machine Learning Research, vol. 80, pp. 5650–5659 (2018). https://doi.org/10.48550/arXiv.1803.01498

[40] Fang, M., Cao, X., Jia, J., Gong, N.Z.: Local model poisoning attacks to Byzantine-robust federated learning. In: 29th USENIX Security Symposium, pp. 1605–1622 (2020). https://doi.org/10.48550/arXiv.1911.11815

[41] Li, S., Ngai, E.C.-H., Voigt, T.: An experimental study of Byzantine-robust aggregation schemes in federated learning. IEEE Transactions on Big Data **10**(6), 975–988 (2024) https://doi.org/10.1109/TBDATA.2023.3237397

[42] Cao, X., Fang, M., Liu, J., Gong, N.Z.: FLTrust: Byzantine-robust federated learning via trust bootstrapping. In: Network and Distributed System Security Symposium (NDSS) (2021). https://doi.org/10.14722/ndss.2021.24434

[43] Bagdasaryan, E., Veit, A., Hua, Y., Estrin, D., Shmatikov, V.: How to backdoor federated learning. In: Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS). Proceedings of Machine Learning Research, vol. 108, pp. 2938–2948 (2020). https://doi.org/10.48550/arXiv.1807.00459

[44] Sun, Z., Kairouz, P., Suresh, A.T., McMahan, H.B.: Can you really backdoor federated learning? arXiv preprint arXiv:1911.07963 (2019) https://doi.org/10.48550/arXiv.1911.07963

[45] Andreina, S., Marson, G.A., Möllering, H., Karame, G.: BaFFLe: Backdoor detection via feedback-based federated learning. In: IEEE 41st International Conference on Distributed Computing Systems (ICDCS), pp. 852–863 (2021). https://doi.org/10.1109/ICDCS51616.2021.00086

[46] Damaskinos, G., El Mhamdi, E.M., Guerraoui, R., Patra, R., Taziki, M.: Asynchronous Byzantine machine learning (the case of SGD). In: Proceedings of the 35th International Conference on Machine Learning (ICML). Proceedings of Machine Learning Research, vol. 80, pp. 1145–1154 (2018). https://doi.org/10.48550/arXiv.1802.07928

[47] Xia, G., Chen, J., Yu, C., Ma, J.: Poisoning attacks in federated learning: A

survey. IEEE Access **11**, 10708–10722 (2023) https://doi.org/10.1109/ACCESS.2023.3238823

[48] Chan, S.-H., Dong, Y., Zhu, J., Zhang, X., Zhou, J.: BadDet: Backdoor attacks on object detection. In: Computer Vision – ECCV 2022 Workshops. Lecture Notes in Computer Science, pp. 396–412. Springer, ??? (2023). https://doi.org/10.1007/978-3-031-25056-9_26

[49] Garg, S., Jönsson, H., Kalander, G., Nilsson, A., Pirange, B., Valadi, V., Östman, J.: Poisoning attacks on federated learning for autonomous driving. In: Proceedings of the Swedish Artificial Intelligence Society Workshop (SAIS). Linköping Electronic Conference Proceedings, vol. 208, pp. 11–18 (2024). https://doi.org/10.3384/ecp208002

[50] Tariq, A., Serhani, M.A., Sallabi, F., Qayyum, T., Barka, E.S., Shuaib, K.A.: Trustworthy federated learning: A survey. arXiv preprint arXiv:2305.11537 (2023) https://doi.org/10.48550/arXiv.2305.11537

[51] Sánchez Sánchez, P.M., Huertas Celdrán, A., Xie, N., Bovet, G., Martínez Pérez, G., Stiller, B.: FederatedTrust: A solution for trustworthy federated learning. Future Generation Computer Systems **152**, 83–98 (2024) https://doi.org/10.1016/j.future.2023.10.013

[52] High-Level Expert Group on Artificial Intelligence: Ethics Guidelines for Trustworthy AI. European Commission, Publications Office of the European Union (2019). https://doi.org/10.2759/346720

[53] European Parliament and Council of the European Union: Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). Official Journal of the European Union, L series, 12 July 2024 (2024). http://data.europa.eu/eli/reg/2024/1689/oj

[54] National Institute of Standards and Technology: Artificial intelligence risk management framework (AI RMF 1.0). Technical Report NIST AI 100-1, NIST (2023). https://doi.org/10.6028/NIST.AI.100-1

[55] Nosek, B.A., Ebersole, C.R., DeHaven, A.C., Mellor, D.T.: The preregistration revolution. Proceedings of the National Academy of Sciences **115**(11), 2600–2606 (2018) https://doi.org/10.1073/pnas.1708274114

[56] Redmon, J., Divvala, S., Girshick, R., Farhadi, A.: You only look once: Unified, real-time object detection. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 779–788 (2016). https://doi.org/10.1109/CVPR.2016.91

[57] Jocher, G., Qiu, J., Chaurasia, A.: Ultralytics YOLO. https://github.com/ultralytics/ultralytics. Software, version 8.4.84 used in this work; no DOI (2025)

[58] Jocher, G., Qiu, J., Liu, M., Lyu, S., Akyon, F.C., Kalfaoglu, M.E.: Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models. arXiv preprint arXiv:2606.03748 (2026). https://doi.org/10.48550/arXiv.2606.03748

[59] Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., Zitnick, C.L.: Microsoft COCO: Common objects in context. In: Computer Vision – ECCV 2014. Lecture Notes in Computer Science, pp. 740–755. Springer, ??? (2014). https://doi.org/10.1007/978-3-319-10602-1_48

[60] Ioffe, S., Szegedy, C.: Batch normalization: Accelerating deep network training by reducing internal covariate shift. In: Proceedings of the 32nd International Conference on Machine Learning (ICML). Proceedings of Machine Learning Research, vol. 37, pp. 448–456 (2015). https://doi.org/10.48550/arXiv.1502.03167

[61] medmabcf: White-Blood-Cell-Detection-Dataset. https://github.com/medmabcf/White-Blood-Cell-Detection-Dataset. Bounding boxes for PBC_dataset_normal_DIB, CC BY 4.0; no DOI (2023)

[62] Acevedo, A., Merino, A., Alférez, S., Molina, Á., Boldú, L., Rodellar, J.: A dataset of microscopic peripheral blood cell images for development of automatic recognition systems. Data in Brief **30**, 105474 (2020) https://doi.org/10.1016/j.dib.2020.105474

[63] Hamamci, I.E., Er, S., Durugol, O.F., Cakmak, G.R., Rosa, E., Simsar, E., *et al.*: DENTEX: Dental enumeration and tooth pathosis detection benchmark for panoramic X-ray. arXiv preprint arXiv:2305.19112 (2023) https://doi.org/10.48550/arXiv.2305.19112

[64] Guerraoui, R., Pinot, R., Rizk, G., Stephan, J., Taiani, F.: Overcoming the Challenges of Batch Normalization in Federated Learning. arXiv preprint arXiv:2405.14670 (2024). https://doi.org/10.48550/arXiv.2405.14670

[65] Bonawitz, K., Ivanov, V., Kreuter, B., Marcedone, A., McMahan, H.B., Patel, S., Ramage, D., Segal, A., Seth, K.: Practical secure aggregation for privacy-preserving machine learning. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 1175–1191 (2017). https://doi.org/10.1145/3133956.3133982

[66] Zhu, L., Liu, Z., Han, S.: Deep leakage from gradients. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 32 (2019). https://doi.org/10.48550/arXiv.1906.08935

[67] Geiping, J., Bauermeister, H., Dröge, H., Moeller, M.: Inverting gradients – how easy is it to break privacy in federated learning? In: Advances in Neural Information Processing Systems (NeurIPS), vol. 33 (2020). https://doi.org/10.48550/arXiv.2003.14053

[68] McMahan, H.B., Ramage, D., Talwar, K., Zhang, L.: Learning differentially private recurrent language models. In: International Conference on Learning Representations (ICLR) (2018). https://doi.org/10.48550/arXiv.1710.06963